

CSC10014 · COMPUTATIONAL THINKING · 2026

Git & GitHub

for Team Projects

A step-by-step tutorial: version control, collaboration, code review and contribution tracking

Practical Track · Lab 1–2 companion · Project: Smart Virtual Assistant

Faculty of Information Technology · University of Science, VNU-HCM

Instructor: Dr. LE Thanh Tung

Contents

How to Use This Tutorial

This tutorial teaches you Git and GitHub from zero to the level you need for the CSC10014 team project: a Smart Virtual Assistant built by a team of **4 students** over eight weeks. It is written for students who can already program (you learned C++ in Fundamentals of Programming) but may never have used version control seriously.

Every chapter follows the same rhythm: a short idea, a picture if it helps, the exact commands to type, the output you should expect, and a small exercise. Do not only read — open a terminal and type along. Git is learned with your fingers, not your eyes.

Who this is for

- Students who have used Git only through "Upload files" on GitHub, or not at all.
- Students who know `add`, `commit` and `push` but get scared by conflicts, rebases or "detached HEAD".
- Team leads who must organise a shared repository, reviews and fair contribution tracking.

What you will be able to do

1. Explain what Git stores and how commits, branches and remotes relate.
2. Create, clone and manage repositories from the command line and on GitHub.
3. Work on branches, open pull requests, review code and resolve merge conflicts calmly.
4. Learn Python in three weeks — coming from C++ — with every exercise tracked in your team repository.
5. Undo almost any mistake — including "I deleted my work".
6. Keep secrets and personal data out of your repository.
7. Measure and present each member's contribution with evidence, fairly.
8. Use AI coding tools with Git safely: small commits, reviews, and an AI-use log.

How the chapters map to the course

Chapter	Topic	Course link
1–3	What Git is, setup, core concepts	Lab 1 (dev environment)
4–7	Daily commands, history, undo, branches	Lab 2 (Git & pull requests)
8–11	GitHub, team workflow, commits, reviews	Weeks 2–5, Milestone 5 (MVP)
12	Tracking contributions	Assessment: contribution factor
13–15	Secrets, AI tools, CI	Weeks 5–6, Labs 8–9
16	Troubleshooting	Every week
17	In-class exercises (team & individual) and how to submit	Labs 1–2
18	3-week Python homework, tracked on GitHub	Weeks 1–3 homework

Chapter	Topic	Course link
19–22	Internals, VS Code, planning tools, tricky files	Going further

Conventions used

Commands you type are shown in grey boxes. Lines starting with **\$** are commands; the other lines are the output you should see. Do not type the **\$**.

```
$ git --version  
git version 2.47.1
```

Throughout the tutorial you will see coloured boxes:

TIP

A shortcut or good habit that saves time.

WARNING

Something that can destroy work or leak secrets. Read these twice.

NOTE

Extra background you can skip on a first reading.

EXERCISE

A hands-on exercise. Do it before moving on.

IN OUR COURSE

A rule or expectation specific to CSC10014.

1. Why Version Control?

1.1 A story every student knows

It is 23:40, the deadline is midnight. Your team has five folders called `project_final`, `project_final2`, `project_final_REAL`, `project_final_Minh_fixed` and `project_final_USE_THIS`. Nobody knows which one contains the working login code. Lan's laptop has the only copy of the database script. Someone overwrote Huy's changes by copying an old folder on top of the new one.

Version control exists to make this story impossible. A version control system (VCS) records every change to a set of files, who made it, when, and why — and lets many people change the same project at the same time without destroying each other's work.

1.2 What Git gives you

- **A complete history.** Every saved version (a *commit*) is kept forever. You can go back to any of them.
- **Safe experiments.** You can try an idea on a *branch*. If it fails, you delete the branch; the main project is untouched.
- **Collaboration.** Four people can work in parallel and combine their work with a *merge*.
- **Accountability.** Every line can be traced to the commit, the person and the message that introduced it.
- **Backup.** Every clone is a full copy of the history. If one laptop dies, nothing is lost.

1.3 Git vs. GitHub

Students often mix these up. They are different things:

	Git	GitHub
What it is	A program on your computer	A website that hosts Git repositories
Works offline?	Yes, completely	No
Main job	Record history, branch, merge	Share repos, pull requests, reviews, issues, CI
Made by	Linus Torvalds, 2005 (for the Linux kernel)	GitHub Inc., now part of Microsoft
Alternatives	Mercurial, SVN (older)	GitLab, Bitbucket, Gitea

You can use Git without GitHub. You cannot really use GitHub without Git. In this course you use both: Git on your laptop, GitHub for the team repository.

1.4 Snapshots, not differences

Many older systems stored the *differences* between versions. Git thinks in *snapshots*: each commit is a picture of the whole project at one moment. Files that did not change are not copied again — Git just points to the previous identical file. This makes switching between versions very fast and makes history very hard to corrupt.

NOTE

Every object Git stores is named by a hash (a SHA-1 or SHA-256 fingerprint) of its content. If a single byte changes, the name changes. That is why Git history is tamper-evident: you cannot silently edit an old commit.

1.5 Why this matters in the AI era

AI coding assistants can now write hundreds of lines in seconds. That makes version control *more* important, not less:

- You need small, reviewable commits so a human can check what the AI produced.
- You need branches so AI-generated experiments never break **main**.
- You need history so you can answer "when did this bug appear, and which change introduced it?"
- You need clear authorship so the team — and the instructor — can see who understood and verified which change.

IN OUR COURSE

In CSC10014 your Git history is part of your grade. The contribution factor (0.7–1.1) is based on evidence from commits, pull requests, reviews and the team logbook. Chapter 12 explains exactly how.

1.6 Check yourself

CHECK YOURSELF

- 1) Name two things Git can do that a shared Google Drive folder cannot.
- 2) Can you commit while offline?
- 3) What is the difference between Git and GitHub in one sentence each?

2. Installing and Configuring Git

2.1 Install Git

Windows

1. Download *Git for Windows* from git-scm.com and run the installer.
2. Keep the defaults, but choose **"Git from the command line and also from 3rd-party software"**.
3. For the default editor, choose **Visual Studio Code** if you have it.
4. For line endings, keep **"Checkout Windows-style, commit Unix-style"**.
5. After installing, open **Git Bash** (or PowerShell) and check the version.

macOS

Open Terminal and type `git --version`. If Git is missing, macOS offers to install the Command Line Tools. Alternatively, with Homebrew: `brew install git`.

Linux (Ubuntu / Debian)

```
$ sudo apt update
$ sudo apt install git
$ git --version
git version 2.43.0
```

TIP

Any Git version from 2.30 onwards works for this tutorial. Commands such as `git switch` and `git restore` need Git 2.23 or newer.

2.2 Tell Git who you are

Every commit records an author name and email. Set them once per computer:

```
$ git config --global user.name "Nguyen Van Minh"
$ git config --global user.email "minh.nv@student.hcmus.edu.vn"
```

IN OUR COURSE

Use your real name and the **same email as your GitHub account**. If the email does not match, GitHub cannot link your commits to your profile and your contribution will look like zero. This is the most common reason students "lose" credit.

Check your settings:

```
$ git config --global --list
user.name=Nguyen Van Minh
user.email=minh.nv@student.hcmus.edu.vn
```

2.3 Useful defaults

```
$ git config --global init.defaultBranch main
$ git config --global core.editor "code --wait"
$ git config --global pull.rebase false
$ git config --global color.ui auto
```

Setting	What it does
<code>init.defaultBranch main</code>	New repositories start with a branch called <code>main</code>
<code>core.editor "code --wait"</code>	Commit messages open in VS Code
<code>pull.rebase false</code>	<code>git pull</code> merges (simplest for beginners)
<code>color.ui auto</code>	Coloured output in the terminal

Line endings

Windows uses CRLF line endings; macOS and Linux use LF. Mixed teams get "every line changed" diffs if this is wrong.

```
# Windows
$ git config --global core.autocrlf true
# macOS / Linux
$ git config --global core.autocrlf input
```

TIP

Add a `.gitattributes` file with the line `* text=auto` to the repository. Then the setting travels with the project, not with each laptop.

2.4 Create a GitHub account

1. Sign up at github.com with a professional username (you will show it to employers).
2. Add your student email under *Settings* → *Emails*, and keep it verified.
3. Turn on **two-factor authentication** (*Settings* → *Password and authentication*).
4. Apply for the free GitHub Student Developer Pack if you want extra tools.

2.5 Connect your laptop to GitHub with SSH

GitHub no longer accepts account passwords for Git operations. Use SSH keys (recommended) or a personal access token.

```
$ ssh-keygen -t ed25519 -C "minh.nv@student.hcmus.edu.vn"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/minh/.ssh/id_ed25519): [Enter]
Enter passphrase (empty for no passphrase): [type a passphrase]
```

Copy the **public** key (the file ending in `.pub`) and add it on GitHub under *Settings* → *SSH and GPG keys* → *New SSH key*:

```
$ cat ~/.ssh/id_ed25519.pub  
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAI... minh.nv@student.hcmus.edu.vn
```

Test the connection:

```
$ ssh -T git@github.com  
Hi minh-nv! You've successfully authenticated, but GitHub does not provide shell access.
```

WARNING

Never share or upload the private key file (`id_ed25519` without `.pub`). Anyone with it can push to your repositories as you.

2.6 Tools that help

- **VS Code** — built-in Git panel, diff viewer, and merge-conflict editor.
- **GitHub CLI** (`gh`) — create pull requests and read issues from the terminal.
- **GitHub Desktop** — a friendly GUI if the terminal feels scary at first. Learn the commands anyway; every GUI is just running them.

EXERCISE

Install Git, set your name and email, create an SSH key, add it to GitHub and run `ssh -T git@github.com`. Paste the success message into your team channel.

3. Core Concepts

Before typing more commands, build a correct mental model. Almost every Git confusion comes from a wrong picture of what Git stores.

3.1 Repository

A *repository* (repo) is a project folder plus a hidden `.git` folder that contains the whole history. Delete `.git` and the folder becomes an ordinary folder with no history.

```
assistant-team07/
├── .git/           # the history lives here – never edit by hand
├── README.md
├── src/
└── tests/
```

3.2 The three areas

Git keeps your work in three places. Understanding them explains `add`, `commit`, `status` and `restore`.

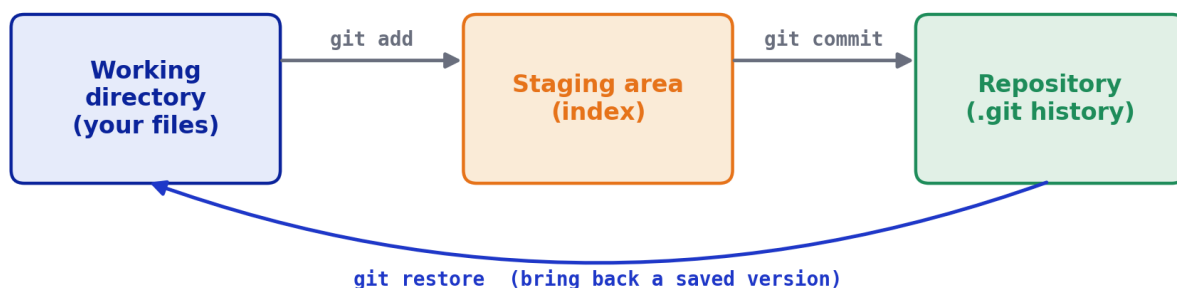


Figure 3.1 — The working directory, the staging area and the repository.

Area	What it is	How things get there
Working directory	The files you see and edit	You edit, create, delete files
Staging area (index)	A draft of the next commit	<code>git add <file></code>
Repository	Saved commits — permanent history	<code>git commit</code>

Why a staging area? Because you often change several things at once, but want to save them as separate, meaningful commits. The staging area lets you choose exactly what goes into the next snapshot.

3.3 Commit

A *commit* is a saved snapshot plus metadata:

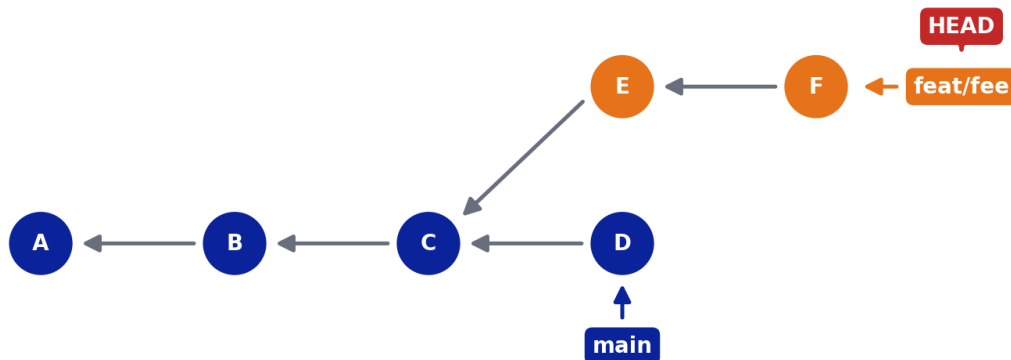
- a unique ID (hash), e.g. `3f9a2c1`
- the author, email and date

- a message explaining *why* the change was made
- a pointer to its *parent* commit(s)

Commits form a chain: each points back to the commit before it. This chain is the project history.

3.4 Branch

A *branch* is simply a movable label that points to one commit. When you commit on a branch, the label moves forward to the new commit. Creating a branch is instant and costs almost nothing — which is why Git users create branches for everything.



Arrows point to the parent commit. A branch is just a movable label on a commit.

Figure 3.2 — Two branches sharing history. `HEAD` marks where you are.

3.5 HEAD

HEAD means "where you are now" — usually the branch you have checked out. When you run `git switch feat/fee`, **HEAD** moves to that branch, and your working directory changes to match its latest commit.

NOTE

"Detached HEAD" means **HEAD** points directly to a commit instead of a branch — for example after `git checkout 3f9a2c1`. It is not dangerous for looking around, but new commits there belong to no branch. Chapter 16 shows how to rescue them.

3.6 Remote

A *remote* is another copy of the repository, usually on GitHub. By convention the main remote is called **origin**. You *push* your commits to it and *fetch* or *pull* other people's commits from it.

3.7 The vocabulary in one table

Term	One-line meaning
repository	project + its full history
working directory	the files you edit
staging area / index	what will go into the next commit
commit	a saved snapshot with a message
hash / SHA	unique ID of a commit

Term	One-line meaning
branch	movable label pointing to a commit
HEAD	where you are now
merge	combine two lines of history
remote / origin	another copy, usually on GitHub
clone	copy a remote repository to your laptop
push / fetch / pull	send / download / download-and-merge commits
pull request (PR)	a GitHub request to merge a branch, with review

CHECK YOURSELF

- 1) After `git add notes.md` but before `git commit`, where is the change?
- 2) What does a branch actually store?
- 3) If you delete the `.git` folder, what do you lose?

4. Your First Repository, Step by Step

In this chapter you create a small repository for the course assistant and practise the daily loop: edit → status → add → commit → log.

4.1 Create a repository

```
$ mkdir assistant-demo
$ cd assistant-demo
$ git init
Initialized empty Git repository in /home/minh/assistant-demo/.git/
```

4.2 Check the status — often

`git status` is the most important command. Run it before and after everything.

```
$ git status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

4.3 Create a file and commit it

Create `README.md` with one line: `# Study Assistant`. Then:

```
$ git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    README.md

$ git add README.md
$ git status
Changes to be committed:
  new file:   README.md

$ git commit -m "docs: add project README"
[main (root-commit) 3f9a2c1] docs: add project README
1 file changed, 1 insertion(+)
create mode 100644 README.md
```

What happened, in terms of the three areas:

1. You created `README.md` in the working directory. Git called it *untracked*.
2. `git add` copied it into the staging area.
3. `git commit` saved the staged snapshot as commit `3f9a2c1`.

4.4 The daily loop

Now add some code. Create `src/priority.py`:

```
def priority(importance: int, days_left: int) -> float:
    """Higher means more urgent."""
    return importance / max(days_left, 1)
```

```
$ git status
Untracked files:
  src/
$ git add src/priority.py
$ git commit -m "feat(planner): add priority rule"
[main 8b1e4d0] feat(planner): add priority rule
1 file changed, 3 insertions(+)
```

Edit the file again — add a docstring line — and look at what changed before committing:

```
$ git diff
diff --git a/src/priority.py b/src/priority.py
@@ -1,3 +1,4 @@
 def priority(importance: int, days_left: int) -> float:
-    """Higher means more urgent."""
+    """Higher means more urgent.
+    days_left < 1 is treated as 1 to avoid division by zero."""
     return importance / max(days_left, 1)
```

Lines starting with `-` were removed; lines starting with `+` were added.

```
$ git add src/priority.py
$ git commit -m "docs(planner): explain days_left guard"
```

4.5 Staging only part of your work

Suppose you edited two files for two different reasons. Commit them separately:

```
$ git status
Changes not staged for commit:
  modified:   src/priority.py
  modified:   README.md

$ git add README.md
$ git commit -m "docs: describe how to run the planner"
$ git add src/priority.py
$ git commit -m "fix(planner): handle negative importance"
```

TIP

`git add -p` lets you stage *parts* of a file, hunk by hunk. It is the best way to keep commits small and focused.

4.6 Look at the history

```
$ git log --oneline
d4c7a90 (HEAD -> main) fix(planner): handle negative importance
a19f3e2 docs: describe how to run the planner
5e02b71 docs(planner): explain days_left guard
8b1e4d0 feat(planner): add priority rule
3f9a2c1 docs: add project README
```

4.7 Ignoring files with .gitignore

Some files must never be committed: virtual environments, caches, secrets, large data. List them in `.gitignore`:

```
# .gitignore for a Python project
.venv/
__pycache__/
*.pyc
.env
data/raw/
*.sqlite
.DS_Store
.vscode/
```

```
$ git add .gitignore
$ git commit -m "chore: add gitignore"
```

WARNING

`.gitignore` only affects files that are *not yet tracked*. If you already committed `.env`, adding it to `.gitignore` does not remove it from history. See Chapter 13.

4.8 Rename and delete files

```
$ git mv src/priority.py src/planner.py
$ git rm old_notes.txt
$ git commit -m "refactor: rename priority module to planner"
```

4.9 A complete first session

Step	Command	Why
1	<code>git status</code>	know where you are
2	edit files	do the work
3	<code>git diff</code>	review your own change
4	<code>git add <files></code> or <code>git add -p</code>	choose what to save
5	<code>git diff --staged</code>	review what will be committed

Step	Command	Why
6	<code>git commit -m "type: message"</code>	save a meaningful snapshot
7	<code>git log --oneline</code>	confirm

EXERCISE

In `assistant-demo`, make five commits: a README, a `.gitignore`, a function, a test for the function, and a fix. Each commit must change one thing only. Show `git log --oneline` to a neighbour and explain every commit in one sentence.

5. Exploring History

History is only useful if you can search it. This chapter shows how to answer questions like "when did this change?", "who wrote this line?" and "what did Lan do this week?".

5.1 git log, many ways

```
$ git log                # full details
$ git log --oneline      # one line per commit
$ git log --oneline --graph --all  # draw branches
$ git log -5            # last 5 commits
$ git log --since="1 week ago"    # recent work
$ git log --author="Lan"         # one person
$ git log -- src/planner.py      # one file
$ git log --stat                # files changed per commit
$ git log -p src/planner.py      # full diffs for one file
```

A readable graph alias:

```
$ git config --global alias.lg "log --oneline --graph --decorate --all"
$ git lg
* 9c1d2e4 (origin/feat/rag) feat(rag): add chunker
| * 71ab0c3 (HEAD -> main, origin/main) Merge pull request #12
|/|
| * 4e5f6a7 feat(fee): add retake rate
|/
* d4c7a90 fix(planner): handle negative importance
```

5.2 git show — one commit in detail

```
$ git show 4e5f6a7
commit 4e5f6a7...
Author: Tran Thi Lan <lan.tt@student.hcmus.edu.vn>
Date:   Tue Oct 6 21:14:03 2026 +0700

    feat(fee): add retake rate

diff --git a/src/fee.py b/src/fee.py
...
```

5.3 Comparing versions with git diff

Command	Compares
<code>git diff</code>	working directory vs. staging area
<code>git diff --staged</code>	staging area vs. last commit
<code>git diff HEAD</code>	working directory vs. last commit
<code>git diff main..feat/fee</code>	two branches
<code>git diff 3f9a2c1 d4c7a90</code>	two commits

Command	Compares
<code>git diff --stat main..feat/fee</code>	summary: files and line counts

5.4 git blame — who wrote this line?

```
$ git blame src/fee.py
4e5f6a7 (Tran Thi Lan 2026-10-06) RETAKE_RATE = 1.5
8b1e4d0 (Nguyen Van Minh 2026-10-01) def tuition(credits, retake=0):
4e5f6a7 (Tran Thi Lan 2026-10-06)     base = credits * RATE
```

NOTE

blame shows the *last* person who touched each line, not necessarily who designed it. Reformatting a file makes you the "author" of every line. Use `git blame -w -C` to ignore whitespace and detect moved code, and never use blame alone to judge contribution (Chapter 12).

5.5 Searching history

```
$ git log --grep="retake"           # commits whose message mentions retake
$ git log -S "RETAKE_RATE"         # commits that added or removed this text
$ git grep "def tuition"           # search the current files
```

`git log -S` is called the *pickaxe*. It is the fastest way to find when a function or constant appeared or disappeared.

5.6 Finding the commit that broke something: git bisect

When a test passed last week but fails today, **bisect** does a binary search over the commits in between:

```
$ git bisect start
$ git bisect bad           # current commit is broken
$ git bisect good v0.3-knowledge # this tag was fine
Bisecting: 6 revisions left to test after this (roughly 3 steps)
$ pytest tests/test_fee.py # run your test
$ git bisect good          # or: git bisect bad
...
4e5f6a7 is the first bad commit
$ git bisect reset
```

With 64 commits between good and bad, bisect needs at most 6 tests. That is Computational Thinking — binary search — applied to debugging.

EXERCISE

In your demo repo, find (1) the commit that created `.gitignore`, (2) every commit that touched `planner.py`, (3) who last changed line 2 of `planner.py`. Write the three commands you used.

6. Undoing Things Safely

Everyone makes mistakes with Git. The good news: almost nothing is truly lost once it has been committed. This chapter gives you a calm, step-by-step way to undo.

6.1 First question: has it been pushed?

Situation	Safe tools
Not committed yet	<code>git restore</code> , <code>git restore --staged</code> , <code>git stash</code>
Committed, not pushed	<code>git commit --amend</code> , <code>git reset</code>
Committed and pushed (shared)	<code>git revert</code> — never rewrite shared history

WARNING

Rewriting history that teammates already pulled (`reset + push --force`) breaks their repositories. On shared branches, always use `git revert`.

6.2 Discard changes in a file you have not staged

```
$ git restore src/planner.py
```

This throws away your uncommitted edits in that file and brings back the last committed version.

6.3 Unstage a file (keep your edits)

```
$ git add .env          # oops
$ git restore --staged .env
```

6.4 Fix the last commit message or add a forgotten file

```
$ git commit -m "feat: add fee calcauator"    # typo
$ git add tests/test_fee.py                  # forgot this
$ git commit --amend -m "feat(fee): add fee calculator and tests"
```

TIP

Only amend commits you have **not pushed** yet.

6.5 Take back the last commit(s) with git reset

`git reset` moves the branch label back to an earlier commit. The three modes differ in what they do to your files.

mode	HEAD / branch	staging area	working directory
--soft	moves HEAD	keeps staging	keeps files
--mixed (default)	moves HEAD	resets staging	keeps files
--hard	moves HEAD	resets staging	OVERWRITES files

Figure 6.1 — What each reset mode touches.

```
$ git reset --soft HEAD~1    # undo commit, keep changes staged
$ git reset HEAD~1          # undo commit, keep changes unstaged (default --mixed)
$ git reset --hard HEAD~1    # undo commit AND delete the changes
```

HEAD~1 means "one commit before HEAD". **HEAD~3** means three commits back.

WARNING

git reset --hard deletes uncommitted work permanently. Run **git status** and **git stash** first if you are not sure.

6.6 Undo a pushed commit with git revert

revert creates a **new** commit that does the opposite of an old one. History stays honest and teammates are not affected.

```
$ git log --oneline -3
71ab0c3 feat(router): route fee questions to LLM
4e5f6a7 feat(fee): add retake rate
d4c7a90 fix(planner): handle negative importance
$ git revert 71ab0c3
[main 2c8e9f1] Revert "feat(router): route fee questions to LLM"
$ git push
```

6.7 Put work aside temporarily with git stash

You are halfway through a change and need to switch branches to fix an urgent bug:

```
$ git stash push -m "wip: chunker experiments"
$ git switch main
# ... fix, commit, push ...
$ git switch feat/rag
$ git stash list
stash@{0}: On feat/rag: wip: chunker experiments
$ git stash pop
```

6.8 The safety net: git reflog

reflog records every position **HEAD** has been in — even after a **reset --hard**. If you "lost" commits, they are almost always here for about 90 days.

```
$ git reset --hard HEAD~3      # disaster: three commits gone
$ git reflog
2c8e9f1 HEAD@{0}: reset: moving to HEAD~3
9a7b6c5 HEAD@{1}: commit: feat(rag): add citations
...
$ git reset --hard 9a7b6c5     # back to where you were
```

6.9 Undo decision table

I want to...	Command
throw away edits in one file	<code>git restore <file></code>
unstage a file	<code>git restore --staged <file></code>
change the last (unpushed) commit	<code>git commit --amend</code>
undo last unpushed commit, keep work	<code>git reset --soft HEAD~1</code>
undo a pushed commit	<code>git revert <hash></code>
park unfinished work	<code>git stash / git stash pop</code>
recover "lost" commits	<code>git reflog</code> then <code>git reset --hard <hash></code> or <code>git branch rescue <hash></code>
get one old version of a file	<code>git restore --source=<hash> <file></code>

EXERCISE

In a scratch repo: make 3 commits, **reset --hard** two of them, then recover them with **reflog**. Then make a commit, push it to a test GitHub repo, and undo it with **revert**. Explain in two sentences why you used **revert** the second time.

7. Branching and Merging

Branches let each teammate work on one feature without disturbing the others. Merging combines the work back. This is the heart of team collaboration.

7.1 Create and switch branches

```
$ git switch -c feat/fee-calculator # create and switch
$ git branch                       # list local branches
* feat/fee-calculator
  main
$ git switch main                  # go back
```

NOTE

Older tutorials use `git checkout -b <name>` and `git checkout <name>`. They still work; `switch` and `restore` were added in Git 2.23 to make the commands clearer.

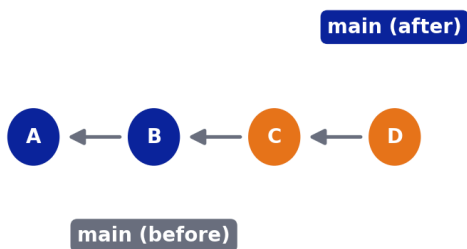
7.2 Branch naming in this course

Prefix	Use for	Example
feat/	new capability	feat/rag-citations
fix/	bug fix	fix/router-empty-message
docs/	documentation	docs/blueprint-diagrams
test/	tests or benchmark	test/eval-50-cases
chore/	tooling, CI, config	chore/ci-ruff

Add the issue number when there is one: `feat/12-fee-calculator`.

7.3 Merging: fast-forward and three-way

Fast-forward merge



Three-way merge

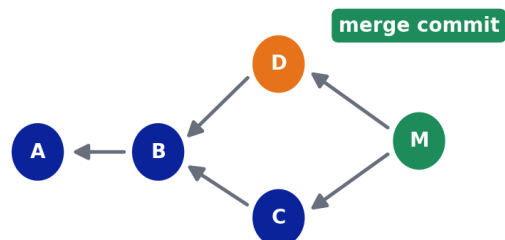


Figure 7.1 — Fast-forward (left) just moves the label; a three-way merge (right) creates a merge commit.

```
$ git switch main
```

```
$ git merge feat/fee-calculator
Updating d4c7a90..4e5f6a7
Fast-forward
 src/fee.py | 12 ++++++++
```

If **main** moved on while you worked, Git creates a merge commit that has two parents:

```
$ git merge feat/rag
Merge made by the 'ort' strategy.
 src/rag.py | 40 ++++++
```

7.4 Merge conflicts — calmly

A conflict happens when two branches changed the same lines differently. Git cannot guess which is right, so it asks you.

Create a conflict on purpose

```
$ git switch -c feat/rate-a
# edit src/fee.py: RATE = 450_000
$ git commit -am "feat(fee): set 2026 rate"
$ git switch main
$ git switch -c feat/rate-b
# edit the same line: RATE = 480_000
$ git commit -am "feat(fee): set new rate"
$ git switch main
$ git merge feat/rate-a      # fine
$ git merge feat/rate-b
CONFLICT (content): Merge conflict in src/fee.py
Automatic merge failed; fix conflicts and then commit the result.
```

Read the conflict markers

```
<<<<<< HEAD
RATE = 450_000
=====
RATE = 480_000
>>>>>> feat/rate-b
```

- Between **<<<<<<** and **=====**: what is on your current branch (**HEAD**).
- Between **=====** and **>>>>>>**: what the incoming branch wants.

Resolve in five steps

1. Open the file. **Talk to the teammate** who wrote the other side if the right answer is not obvious.
2. Edit the file so it contains the correct final code, and delete all three marker lines.
3. Run the tests.
4. **git add src/fee.py**
5. **git commit** (Git proposes a merge message; keep it or improve it).

```
RATE = 480_000  # confirmed with Finance notice 26-07
```

```
$ git add src/fee.py
$ git commit
[main 5a6b7c8] Merge branch 'feat/rate-b'
```

TIP

VS Code shows "Accept Current / Accept Incoming / Accept Both" buttons above each conflict. They are convenient — but you still must read the result and run the tests.

WARNING

Never commit a file that still contains <<<<<< or >>>>>>. Add a CI check or pre-commit hook that fails if these markers appear.

Abort if you are lost

```
$ git merge --abort
```

7.5 Avoiding conflicts

- Pull `main` into your branch often (daily).
- Keep branches short-lived: ideally merged within 1–2 days.
- Split work by module, following your interface contracts (Part 3 of the project slides).
- Do not reformat whole files in the same PR as a logic change.
- Agree on a formatter (e.g. `ruff format`) so style never causes conflicts.

7.6 Rebase — an introduction

`git rebase main` replays your branch's commits on top of the latest `main`, producing a straight history.

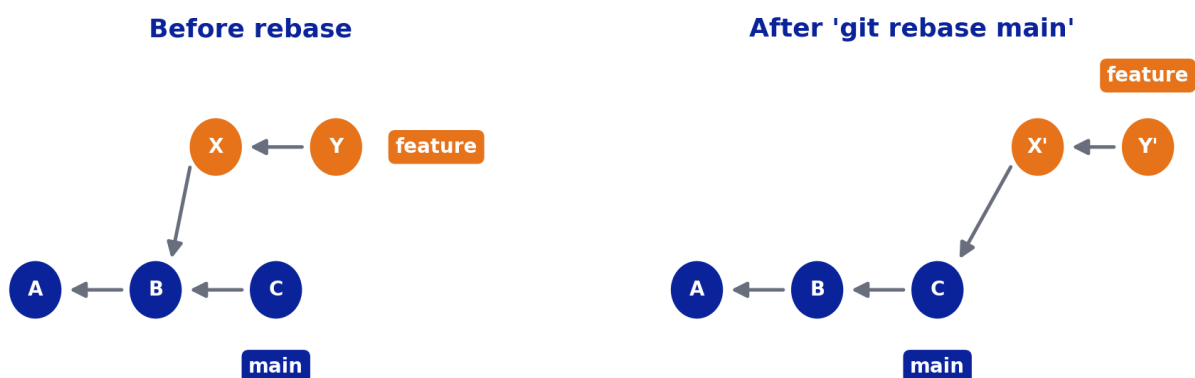


Figure 7.2 — Rebase rewrites X and Y as new commits X' and Y' on top of main.

```
$ git switch feat/rag
$ git fetch origin
$ git rebase origin/main
# if a conflict appears: fix, git add, then
$ git rebase --continue
# or give up:
$ git rebase --abort
```

	Merge	Rebase
History	true, with merge commits	straight, rewritten
Safe on shared branches?	yes	no — only on your own branch
Conflicts	once	possibly once per commit
Recommended here	default	optional, for your own feature branch

IN OUR COURSE

The golden rule: **never rebase a branch someone else is working on**, and never rebase **main**. If in doubt, merge.

7.7 Cleaning up branches

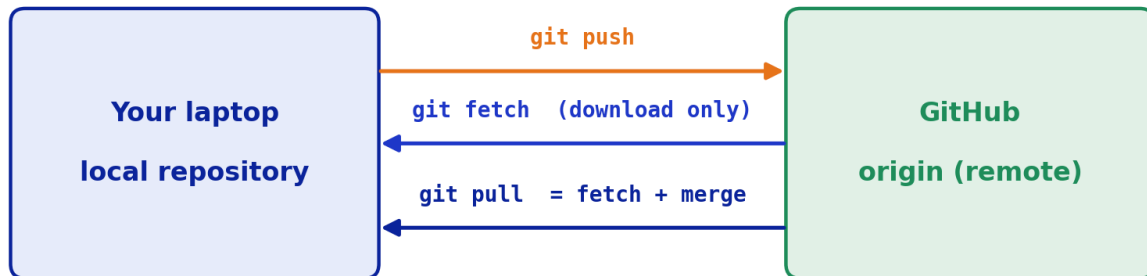
```
$ git branch -d feat/fee-calculator      # delete merged branch
$ git branch -D feat/failed-experiment   # force delete unmerged branch
$ git push origin --delete feat/fee-calculator
$ git fetch --prune                      # forget deleted remote branches
```

EXERCISE

With a partner, create a conflict on purpose in a shared file, resolve it together, and write in the merge commit message *why* you chose the final version.

8. Working with GitHub

8.1 Local and remote repositories



git clone = copy the whole remote repository once

Figure 8.1 — push, fetch and pull between your laptop and GitHub.

8.2 Create the team repository

One person (the Tech Lead) does this:

1. On GitHub, create an **organisation** for the team (free), e.g. `csc10014-team07`.
2. Create a repository `assistant-team07` inside it. Choose **Private** unless your instructor asks otherwise. Add a README and a Python `.gitignore`.
3. *Settings* → *Collaborators and teams*: add all members with **Write** access, and the instructor/TA as required.
4. Protect `main` (Section 9.6).

8.3 Clone

```

$ git clone git@github.com:csc10014-team07/assistant-team07.git
$ cd assistant-team07
$ git remote -v
origin  git@github.com:csc10014-team07/assistant-team07.git (fetch)
origin  git@github.com:csc10014-team07/assistant-team07.git (push)

```

8.4 Push your branch

```

$ git switch -c feat/12-fee-calculator
# ... commits ...
$ git push -u origin feat/12-fee-calculator

```

`-u` sets the *upstream*: afterwards a plain `git push` or `git pull` on this branch knows where to go.

8.5 Fetch vs. pull

```
$ git fetch origin          # download, change nothing locally
$ git log --oneline main..origin/main  # what is new on GitHub?
$ git pull                  # fetch + merge into current branch
```

TIP

When you are unsure, **fetch** first and look. **pull** is **fetch** plus **merge**, and merges can conflict.

8.6 Keep your feature branch up to date

```
$ git switch feat/12-fee-calculator
$ git fetch origin
$ git merge origin/main      # or: git rebase origin/main (your own branch only)
$ git push
```

8.7 When push is rejected

```
$ git push
! [rejected]        main -> main (fetch first)
error: failed to push some refs
```

Someone pushed before you. Do **not** force-push. Instead:

```
$ git pull          # merge their work
# resolve conflicts if any, run tests
$ git push
```

8.8 Forks (for open-source contributions)

For the team project everyone pushes branches to the same repository. For open-source projects where you have no write access, you *fork* (make your own copy on GitHub), push to the fork, and open a pull request to the original.

```
$ git remote add upstream git@github.com:original/project.git
$ git fetch upstream
$ git merge upstream/main
```

8.9 Tags and releases

Milestones in this course are marked with tags:

```
$ git switch main
$ git pull
$ git tag -a v0.1-brief -m "Milestone 1: Project Brief v1"
$ git push origin v0.1-brief
```

Tag	Milestone
v0.1-brief	M1 Project Brief
v0.2-blueprint	M2 System Blueprint
v0.3-knowledge	M3 Knowledge Layer
v0.4-prototype	M4 Capability Prototype
v0.5-mvp	M5 MVP v1
v0.6-risk	M6 Failure & Risk Pack
v0.7-eval	M7 Evaluation Report
v1.0	M8 Final Product

IN OUR COURSE

The instructor grades the state of the repository **at each tag**. A tag pushed after the deadline, or moved later, is visible in GitHub's history.

9. The Team Workflow

This chapter turns the commands into a team process. It follows *GitHub Flow*, a simple and widely used model: **main** is always working; every change goes through a short-lived branch and a pull request.



Figure 9.1 — The life of one change.

9.1 Recommended repository structure

```

assistant-team07/
├── README.md           # what, why, how to run (10-minute test)
├── AGENTS.md           # rules for AI coding agents (Chapter 14)
├── CONTRIBUTING.md     # team rules: branches, commits, reviews
├── .github/
│   ├── pull_request_template.md
│   ├── ISSUE_TEMPLATE/
│   ├── CODEOWNERS
│   └── workflows/ci.yml
├── docs/               # brief, blueprint, ADRs, reports
│   └── adr/
├── src/                # application code
├── tests/              # unit & integration tests
├── eval/               # benchmark and evaluation scripts
├── data/               # small, non-sensitive samples only
├── .env.example        # config template – never the real .env
└── .gitignore
  
```

9.2 Step 1 — Plan work as issues

Every piece of work starts as a GitHub issue with a clear title, a short description, acceptance criteria and an assignee.

```

Title: Fee calculator for retake credits (#12)
Description: UC-02 needs tuition for N credits including retakes.
Acceptance criteria:
- [ ] calc_fee(credits, retake) returns VND as int
- [ ] rejects credits outside 1-30
- [ ] 3 unit tests incl. invalid input
Assignee: @lan-tt  Labels: feat, planner  Milestone: M4
  
```

Use a **Project board** (Todo / In progress / In review / Done) and GitHub **Milestones** matching M1–M8.

9.3 Step 2 — Branch from an up-to-date main

```
$ git switch main
$ git pull
$ git switch -c feat/12-fee-calculator
```

9.4 Step 3 — Commit in small steps and push

```
$ git add src/fee.py
$ git commit -m "feat(fee): add calc_fee with retake rate"
$ git add tests/test_fee.py
$ git commit -m "test(fee): cover invalid credit ranges"
$ git push -u origin feat/12-fee-calculator
```

9.5 Step 4 — Open a pull request

On GitHub click **Compare & pull request**, or use the CLI:

```
$ gh pr create --title "feat(fee): fee calculator (#12)" --body-file
.github/pull_request_template.md
```

A good PR description answers: what, why, how it was tested, and how AI tools were used.

```
## What
Adds calc_fee(credits, retake) for UC-02.
## Why
Closes #12. Fees must be exact; the LLM only phrases the result (ADR-0002).
## How tested
pytest tests/test_fee.py - 5 tests, all pass.
## AI use
Copilot suggested the validation block; I rewrote the error messages
and added the negative-credit test myself.
## Checklist
- [x] tests pass locally - [x] no secrets - [x] docs updated
```

TIP

Writing **Closes #12** in the PR description automatically closes issue 12 when the PR is merged — and links the work to the issue for contribution tracking.

9.6 Step 5 — Protect main

As repository admin: *Settings* → *Branches* → *Add branch protection rule* (or *Rulesets*) for **main**:

- Require a pull request before merging.
- Require at least **1 approval**.
- Require status checks (CI) to pass.
- Block force pushes and deletion.
- Optionally: require conversation resolution before merging.

Now nobody — including the Tech Lead — can push directly to **main**. Every change leaves a reviewed, traceable PR.

9.7 Step 6 — Review, then merge

At least one teammate reviews (Chapter 11). When approved and CI is green, merge. Three merge options exist on GitHub:

Option	Result	When
Create a merge commit	keeps all commits + a merge commit	default, full history
Squash and merge	one commit on main per PR	clean history; recommended for this course
Rebase and merge	commits replayed linearly	advanced teams

NOTE

With *squash and merge*, GitHub keeps the author of the PR as the commit author and adds **Co-authored-by** lines for other contributors. Your detailed commits remain visible inside the PR.

9.8 Step 7 — Clean up and sync

```
$ git switch main
$ git pull
$ git branch -d feat/12-fee-calculator
```

9.9 A week in the life of the team

Day	Activity	Git evidence
Mon (studio)	sprint planning	issues created & assigned, milestone set
Tue–Wed	build	branches, small commits, draft PRs
Wed	mid-week sync (15 min)	PR comments, board updated
Thu–Fri	review & merge	approvals, merged PRs, CI runs
Sun	milestone evidence	tag pushed, docs updated, logbook entry

9.10 Team roles and Git responsibilities

Role	Git responsibility
Tech Lead	repo settings, protection rules, CI, final merges at milestones
Product Lead	issues, milestones, board hygiene
Every member	own branches, PRs, at least one review per week

Role	Git responsibility
QA & Security Lead	tests, <code>eval/</code> folder, CI quality gate, secrets policy
Knowledge & AI Engineer	<code>src/rag/</code> , prompts, <code>docs/ai-use-log.md</code>

IN OUR COURSE

Rotate who merges and who reviews. With only four members, every person must be visible. A repository where one person made 90% of commits and all merges is a red flag in assessment — even if the others "helped verbally".

EXERCISE

As a team: create the organisation and repository, add a README, `.gitignore`, `CONTRIBUTING.md` and PR template, protect `main`, and have **every** member merge one small PR (e.g. adding their name and role to `docs/team.md`) reviewed by someone else.

10. Writing Good Commits

A commit history is a story your future self, your teammates and your instructor will read. Good commits make reviews fast, bugs easy to find and contributions easy to see.

10.1 One commit, one idea

A good commit is **atomic**: it does one logical thing, and the project still works after it.

Bad	Good
"fix stuff" — 14 files, a feature, two bug fixes and reformatting	three commits: <code>feat(fee)...</code> , <code>fix(router)...</code> , <code>style: run formatter</code>
"WIP" pushed at 2 a.m. with failing tests	small working steps, each with passing tests
"update"	"fix(planner): treat days_left < 1 as 1"

10.2 The commit message format

We use the *Conventional Commits* style:

```
<type>(<scope>): <short summary in imperative mood>
<optional body: why the change was needed, what alternatives were rejected>
<optional footer: Closes #12, Co-authored-by: ...>
```

Type	Meaning
<code>feat</code>	a new capability for users
<code>fix</code>	a bug fix
<code>docs</code>	documentation only
<code>test</code>	adding or fixing tests / benchmark cases
<code>refactor</code>	restructure code, no behaviour change
<code>perf</code>	faster or cheaper, same behaviour
<code>style</code>	formatting only
<code>chore</code>	tooling, dependencies, CI

10.3 Examples from the assistant project

```
feat(router): add clarify path when deadline is missing

The parser returns NeedsClarification when no date is found.
Previously the router sent these to the LLM, which guessed dates.

Closes #27
```

```
fix(rag): keep rule and exception in the same chunk
```

Chunking by 400 tokens split handbook §4.2 in two, so the exception "except final-year students" was never retrieved. Now we chunk by section heading. Recall@5 on dev set: 62% -> 81%.

```
test(eval): add 15 adversarial benchmark cases from red team
```

10.4 Rules of thumb

- Summary line ≤ 72 characters, imperative mood ("add", not "added" or "adds").
- The body explains **why**, not what — the diff already shows what.
- Reference issues: **Closes** #12, **Refs** #30.
- Commit when tests pass. Push at least once a day.
- Never commit generated files, virtual environments, secrets or large datasets.

10.5 Giving credit: Co-authored-by

When you pair-program, or a teammate substantially helped, add a trailer at the end of the message:

```
feat(ui): add feedback thumbs to chat view
```

```
Co-authored-by: Tran Thi Lan <lan.tt@student.hcmus.edu.vn>
```

GitHub shows both avatars on the commit and counts it for both people.

IN OUR COURSE

Pair programming is encouraged. Use **Co-authored-by** so both partners get visible credit — the email must match the partner's GitHub email.

10.6 Commit hooks (optional)

A *pre-commit hook* runs checks before every commit, e.g. formatting, linting and secret detection. The **pre-commit** framework makes this easy:

```
$ pip install pre-commit
# .pre-commit-config.yaml
repos:
- repo: https://github.com/astral-sh/ruff-pre-commit
  rev: v0.6.9
  hooks: [{id: ruff}, {id: ruff-format}]
- repo: https://github.com/gitleaks/gitleaks
  rev: v8.21.2
  hooks: [{id: gitleaks}]
$ pre-commit install
```

EXERCISE

Rewrite these messages in Conventional Commits style: "fixed bug", "new stuff for rag", "Update README.md", "final version". For each, invent a plausible body explaining why.

11. Code Review on GitHub

A pull request is where teamwork becomes visible. Reviews catch bugs, spread knowledge and — in this course — are evidence of contribution.

11.1 Why review?

- Two people understand every change (bus factor > 1).
- Bugs and security problems are cheaper to fix before merging.
- AI-generated code gets a human check.
- Reviewers learn the codebase; authors learn better practices.

11.2 How to review a PR, step by step

1. Read the PR description and the linked issue. Do you understand the goal?
2. Open *Files changed*. Start with tests: what behaviour do they promise?
3. Read the code for **design** (right place? right boundary?), **correctness**, **complexity**, **naming**, **tests** and **docs**.
4. Check out the branch and run it if the change is non-trivial:

```
$ gh pr checkout 34
$ pytest -q
```

5. Leave comments, then submit the review as **Comment**, **Approve** or **Request changes**.

11.3 Writing review comments

Label comments so authors know what matters:

Prefix	Meaning
Blocking:	must be fixed before merge
Question:	I do not understand — please explain
Suggestion:	optional improvement
Nit:	tiny style issue, optional
Praise:	something done well — say it!

Unhelpful	Helpful
"This is wrong."	"Blocking: <code>calc_fee(0)</code> returns 0 — credits must be 1–30 per UC-02. Raise <code>ValueError</code> ?"
"Why??"	"Question: why is the retake rate hard-coded instead of read from <code>fees.csv</code> ?"
"LGTM" on 900 lines	"Could we split this into the router change and the UI change? Hard to review as one."

11.4 Suggested changes

GitHub lets reviewers propose exact code. Select lines in *Files changed*, click the $\pm$ icon, and write a block that starts with three backticks followed by the word **suggestion** and ends with three backticks. Inside it, put the replacement code:

```
(three backticks)suggestion
    if not 1 <= credits <= 30:
        raise ValueError("credits must be between 1 and 30")
(three backticks)
```

The author can accept it with one click; it becomes a commit co-authored by the reviewer.

11.5 Responding as an author

- Reply to every comment: fix it, or explain politely why not.
- Push fixes as new commits (do not force-push during review — reviewers lose their place).
- Click **Resolve conversation** when a thread is done.
- Re-request review when ready.

11.6 Reviewing AI-generated code

AI assistants produce plausible code quickly. Reviewers should check a few extra things:

- **Imports:** does every package really exist, and is it needed? (AI tools sometimes invent names; attackers register them.)
- **Secrets & data:** no API keys, no real student data in examples.
- **Tests:** were tests added — or weakened or deleted to make them pass?
- **Over-engineering:** factories, abstract classes or configs for one use?
- **Edge cases:** empty input, zero, very large values, Vietnamese diacritics.
- **Explainability:** can the author explain every line when asked?

IN OUR COURSE

Every PR description has an **AI use** section. "None" is a valid answer. Undisclosed AI-generated code is an integrity issue; disclosed and verified AI use is welcome.

11.7 Review etiquette

- Review within 24 hours. A PR waiting three days blocks the whole team.
- Review the code, not the person.
- Keep PRs small: aim for 100–300 changed lines. Larger PRs get worse reviews.
- Approve only what you actually read and understood.

EXERCISE

Pair up. Each person opens a small PR. Review your partner's PR with at least one Question, one Suggestion (using a suggestion block) and one Praise. Resolve all threads and merge.

12. Tracking Contributions

In a team of 4, fair assessment needs evidence. Git and GitHub record a lot of it — but raw numbers can mislead. This chapter shows how to collect evidence, how to read it critically, and how the course uses it.

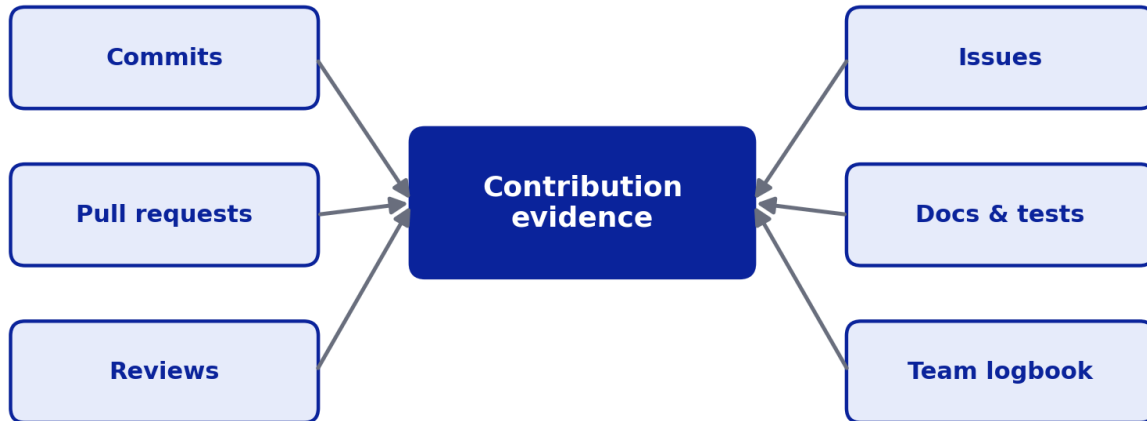


Figure 12.1 — Contribution evidence comes from several sources, not only commit counts.

12.1 What counts as contribution?

Kind	Visible in
Code & tests	commits, PRs, lines changed in <code>src/</code> , <code>tests/</code>
Reviews	PR reviews, review comments, suggestions
Design & docs	commits in <code>docs/</code> , ADRs, diagrams
Planning & coordination	issues opened, board updates, meeting notes
Evaluation	benchmark cases in <code>eval/</code> , error analysis
Pair work	<code>Co-authored-by</code> trailers

NOTE

Lines of code is a weak measure. A 3-line fix after two hours of debugging can matter more than 500 lines of generated boilerplate. Always combine several signals and read the actual work.

12.2 Make sure your work is attributed to you

1. Your `user.email` in Git must be an email listed in your GitHub account (Section 2.2).
2. Check recent commits: `git log --format="%an <%ae>" -10`.
3. If you committed with a wrong email, the team can map identities with a `.mailmap` file (Section 12.6).
4. When pairing, add `Co-authored-by`.
5. Open PRs from your own account; do not let one person push everyone's code.

12.3 Command-line evidence

Commits per person

```
$ git shortlog -sne --no-merges
42  Tran Thi Lan <lan.tt@student.hcmus.edu.vn>
35  Nguyen Van Minh <minh.nv@student.hcmus.edu.vn>
28  Le Quoc Huy <huy.lq@student.hcmus.edu.vn>
19  Pham Mai <mai.p@student.hcmus.edu.vn>
```

Commits in a period (e.g. one milestone)

```
$ git shortlog -sn --no-merges v0.4-prototype..v0.5-mvp
$ git log --since="2026-10-12" --until="2026-10-19" --no-merges --oneline --author="Mai"
```

Lines added and removed per person

```
$ git log --no-merges --author="Lan" --pretty=tformat: --numstat \
| awk '{add+=$1; del+=$2} END {print "added:", add, "removed:", del}'
added: 1840 removed: 612
```

Which files did someone work on?

```
$ git log --no-merges --author="Huy" --name-only --pretty=format: | sort | uniq -c | sort -rn
| head
14  src/router.py
9   tests/test_router.py
5   docs/adr/0004-routing.md
```

Work by folder (code vs. tests vs. docs)

```
$ for d in src tests docs eval; do
  echo "== $d"; git shortlog -sn --no-merges -- $d; done
```

Current ownership of lines

```
$ git ls-files 'src/*.py' | xargs -n1 git blame -w -C --line-porcelain \
| grep "^author " | sort | uniq -c | sort -rn
```

WARNING

blame counts who last touched a line. A formatter run or a file move can "steal" thousands of lines. Use **-w** **-C** and treat the result as a rough picture only.

12.4 GitHub Insights

On the repository page, open **Insights**:

- **Contributors** — commits and additions/deletions per person over time. Only counts the default branch and linked emails.
- **Pulse** — merged PRs, opened/closed issues and active authors in the last week or month.
- **Commits and Code frequency** — activity over time. A flat line followed by a spike the night before a deadline is visible to everyone.
- **Network** — the branch graph.

12.5 Pull requests, reviews and issues with the GitHub CLI

```
$ gh pr list --state merged --limit 200 --json author,number,title \
  --jq '.[[] | .author.login' | sort | uniq -c | sort -rn
    14 lan-tt
    11 minh-nv
     9 huy-lq
     7 mai-p

$ gh search prs --repo csc10014-team07/assistant-team07 --reviewed-by mai-p --json number | jq
length
12

$ gh issue list --state all --limit 300 --json author --jq '.[].author.login' | sort | uniq -c
```

Reviews are real contribution. In the example, Mai authored the fewest PRs but reviewed twelve — the evidence tells a more complete story than commit counts alone.

12.6 Fixing identities with .mailmap

If Minh committed from two emails, create **.mailmap** in the repository root:

```
Nguyen Van Minh <minh.nv@student.hcmus.edu.vn> <minh@laptop.local>
Nguyen Van Minh <minh.nv@student.hcmus.edu.vn> <nvminh2004@gmail.com>
```

git shortlog and **git blame** now merge the identities.

12.7 A contribution report script

Save as **tools/contrib_report.py** and run it before each milestone. It combines commits, lines, files by area and co-authored commits into one table.

```
#!/usr/bin/env python3
"""Contribution report between two refs, e.g. v0.4-prototype..v0.5-mvp."""
import collections, re, subprocess, sys

rng = sys.argv[1] if len(sys.argv) > 1 else "HEAD"
log = subprocess.run(
    ["git", "log", rng, "--no-merges", "--use-mailmap", "--numstat",
     "--format=@%aN|%b"], capture_output=True, text=True, check=True).stdout

stats = collections.defaultdict(lambda: collections.Counter())
author = None
for line in log.splitlines():
    if line.startswith("@@"):
        author, body = line[2:].split("|", 1)
        stats[author]["commits"] += 1
```

```

    for co in re.findall(r"Co-authored-by: ([^<]+)<", body):
        stats[co.strip()][ "co_authored" ] += 1
    elif line.strip() and author:
        add, dele, path = line.split("\t")
        if add == "-":
            # binary file
            continue
        area = path.split("/")[0] if "/" in path else "root"
        stats[author][ "added" ] += int(add)
        stats[author][ "removed" ] += int(dele)
        stats[author][ f"files_{area}" ] += 1

areas = [ "src", "tests", "docs", "eval" ]
print(f"{'author':22} commits  added removed co-auth " + " ".join(f"{a:>6}" for a in areas))
for a, c in sorted(stats.items(), key=lambda kv: -kv[1][ "commits" ]):
    print(f"{a:22} {c[ 'commits' ]:7} {c[ 'added' ]:6} {c[ 'removed' ]:7} {c[ 'co_authored' ]:7} "
          + " ".join(f"{c[ 'files_' + x ]:6}" for x in areas))

```

```

$ python tools/contrib_report.py v0.4-prototype..v0.5-mvp
author           commits  added removed co-auth   src  tests  docs  eval
Tran Thi Lan      18    920    210     2    22    14     3     0
Nguyen Van Minh   15    640    180     1    19     6     8     0
Le Quoc Huy       12    510     95     0    11     9     2     4
Pham Mai          9    210     40     3     2     3    12     9

```

NOTE

Note in the example how Mai contributes mostly through docs and evaluation. A table by area — plus review counts from Section 12.5 — gives a fair picture. A single "commits" column does not.

12.8 Reading the evidence critically

Pattern	Possible meaning	What to check
One person has 80% of commits	others do not code — or one person commits everyone's work	PR authors, co-authors, logbook
Huge line counts	generated code, vendored libraries, data files	what files? were they reviewed?
All commits on the last night	no iteration, rushed work	milestone history, weekly logs
Many tiny commits ("fix", "fix2")	commit-count gaming or messy work	commit quality, PR descriptions
Few commits but many reviews	reviewer / tester role	review comments' quality
Zero activity for two weeks	disengagement or unlinked email	email check, logbook, talk to them

WARNING

Do not game the numbers. Splitting one change into 20 commits or committing generated files is easy to spot and counts as poor practice, not contribution.

12.9 How the course uses contribution evidence

Your individual project score is the team score multiplied by a contribution factor (typically 0.7–1.1). The factor is set from several sources together:

1. **Repository evidence** — commits, merged PRs, reviews, issues, co-authored work (this chapter).
2. **Peer evaluation** — confidential ratings from teammates at Week 4 and Week 8.
3. **Team logbook** — weekly task allocation and who did what.
4. **Individual defense** — can you explain the parts you claim, including AI-assisted code?

IN OUR COURSE

Green flags: steady weekly activity, reviewed PRs from every member, thoughtful reviews, tests and docs for your own work. Red flags: no merged PR by Week 5, cannot explain own code, undisclosed AI-generated work, all activity in the final days.

12.10 The team logbook

Keep `docs/logbook.md` updated every week. It captures work Git cannot see — interviews, design sessions, meetings.

```
## Week 5 (12–18 Oct) – Sprint goal: MVP end-to-end
| Member | Planned | Done (evidence) | Hours |
|-----|-----|-----|-----|
| Lan | fee tool + tests | PR #34, #36 merged | 7 |
| Minh | router clarify path | PR #35; reviewed #34 | 6 |
| Huy | CI + docker compose | PR #37; fixed CI in #38 | 5 |
| Mai | README + 20 eval cases | PR #39, #40; reviewed #35, #37 | 7 |
Blockers: LLM rate limit on shared key → Huy adds caching (#41).
AI use: Copilot for fee validation (Lan, disclosed in #34).
```

EXERCISE

Run `git shortlog -sne --no-merges` and the contribution script on your team repository. Discuss in the team: does the table match reality? What is missing? Add anything missing to the logbook.

13. Secrets, Privacy and Large Files

13.1 Never commit secrets

API keys, passwords, tokens and database URLs must never enter Git history. Bots scan public GitHub for keys within minutes; private repositories leak too (forks, screenshots, a teammate's compromised account).

Use environment variables and a `.env` file that is ignored:

```
# .gitignore
.env
.env.*
!.env.example
```

```
# .env.example (committed)
LLM_API_KEY=changeme
LLM_PROVIDER=openai
DB_URL=sqlite:///dev.db
```

```
# src/settings.py
import os
from dotenv import load_dotenv
load_dotenv()
LLM_API_KEY = os.environ["LLM_API_KEY"]
```

13.2 If you leaked a secret

1. **Revoke or rotate the key immediately** at the provider. This is the only step that really protects you.
2. Tell your team and instructor.
3. Remove it from the current files and commit.
4. If the repository is public or the key is sensitive, remove it from history with `git filter-repo` (or GitHub's guidance on removing sensitive data), then force-push *in coordination with the whole team*.
5. Turn on **secret scanning** and **push protection** (*Settings* → *Code security*), and add a `gitleaks` pre-commit hook.

```
$ pip install git-filter-repo
$ git filter-repo --replace-text <(echo 'sk-abc123...==>REMOVED')
```

WARNING

Deleting the file in a new commit does **not** remove the secret — it is still in the history. Rotate the key first, always.

13.3 Personal data

The assistant may handle student names, IDs, grades or messages. Rules for the course:

- Never commit real personal data. Use synthetic data (`data/sample_students.csv` with invented names).
- Never commit raw chat logs from real users. Store anonymised extracts only, and only if needed.
- Interview notes in `docs/research/` must not contain names, phone numbers or student IDs.
- Follow Vietnam's Personal Data Protection Law (Law 91/2025/QH15, in force from 1 January 2026) and the faculty's rules.

13.4 Large files and datasets

Git is designed for text. Large binary files (videos, model weights, big PDFs, datasets) bloat the repository forever.

- Keep only small sample data in Git (< 1–5 MB).
- Put large data in cloud storage or a shared drive; document where in `data/README.md`.
- If you must version large files, use **Git LFS**:

```
$ git lfs install
$ git lfs track "*.pdf"
$ git add .gitattributes
```

TIP

GitHub warns at 50 MB and blocks files over 100 MB. If `git push` is suddenly very slow, check for an accidentally committed dataset: `git rev-list --objects --all | git cat-file --batch-check | sort -k3 -n | tail`.

13.5 Checklist before every push

Check	Command
No secrets	<code>git diff --staged</code> and look; <code>gitleaks protect --staged</code>
No personal data	review <code>data/</code> and <code>docs/research/</code> changes
No large files	<code>git status</code> , file sizes
Tests pass	<code>pytest -q</code>
No conflict markers	<code>git grep -n "<<<<<<<<"</code>

14. Git with AI Coding Tools

AI assistants (Copilot, Claude Code, Cursor, ChatGPT and others) can write code, commit messages and even open pull requests. Git is what keeps that power safe.

14.1 Principles

- **You own every line you merge**, whoever or whatever wrote it.
- **Small diffs.** Ask the AI for one function or one change at a time, commit it, review it.
- **Branches for experiments.** Let an agent work on `exp/agent-refactor`, never on `main`.
- **Tests first.** Write or agree on tests, then ask the AI to make them pass — and check it did not edit the tests.
- **Disclose.** Record AI use in the PR description and the AI-use log.

14.2 A safe AI-assisted loop

1. Create a branch: `git switch -c feat/31-next-week`.
2. Write the tests (or ask the AI to propose tests, then review them yourself).
3. Ask the AI for the implementation.
4. Read the diff: `git diff`. Do you understand every line?
5. Run tests and linters.
6. Commit with a clear message; push; open a PR with an AI-use note.
7. A teammate reviews with the checklist in Section 11.6.

14.3 Check what the agent changed

```
$ git status           # which files changed?
$ git diff --stat      # how much?
$ git diff tests/      # did it touch the tests?
$ git diff -- '*.toml' '*.txt' requirements* # new dependencies?
```

WARNING

If an agent says "all tests pass", check that tests were not deleted, skipped (`@pytest.mark.skip`) or weakened. Read the test diff first.

14.4 AGENTS.md — rules for agents in your repo

Many coding agents read an `AGENTS.md` file in the repository root. Put your team rules there:

```
# AGENTS.md
- Setup: uv sync; cp .env.example .env
- Test: pytest -q must pass before any commit
- Style: ruff check && ruff format; type hints required
- Never commit .env, real student data, or files in data/raw/
- Never push to main; work on a feature branch and open a PR
- Fees and eligibility rules live only in src/tools/, never in prompts
- Do not add new dependencies without asking
```

14.5 AI-written commit messages

AI tools can draft commit messages from a diff. Treat drafts like any other AI output: check that the message is true, explains *why*, and references the issue. "Update files" written by an AI is still a bad message.

14.6 The AI-use log

Keep `docs/ai-use-log.md`, one row per significant use:

Date	Who	Tool	Asked for	Kept / changed	Verified by
12/10	Lan	Copilot	fee validation	rewrote messages	5 unit tests
14/10	Minh	Claude	router tests	removed 3 duplicates	failed on old bug first
15/10	Mai	ChatGPT	README draft	Windows steps added	teammate ran setup

IN OUR COURSE

The AI-use log is part of the team logbook deliverable. It protects you: disclosed, verified AI use is fine; undisclosed AI output is an integrity issue.

15. Continuous Integration with GitHub Actions

CI runs your checks automatically on every push and pull request, so **main** stays green.

15.1 A first workflow

Create `.github/workflows/ci.yml`:

```
name: CI
on:
  pull_request:
  push:
    branches: [main]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r requirements.txt
      - run: ruff check .
      - run: pytest -q
      - name: No conflict markers
        run: "! git grep -n -E '^(<<<<<<|>>>>>>)'"
```

15.2 Make CI required

In the branch protection rule for **main**, tick **Require status checks to pass** and select the **test** job. Now a red CI blocks merging.

15.3 Using secrets in CI

Never put keys in the workflow file. Add them under *Settings* → *Secrets and variables* → *Actions* and reference them:

```
- run: pytest -q tests/integration
  env:
    LLM_API_KEY: ${ secrets.LLM_API_KEY }
```

TIP

Keep most tests independent of real APIs by using a fake LLM (adapter pattern). Run the few real-API tests only on **main** or on a schedule to save cost.

15.4 Reading a failed run

1. Open the PR → *Checks* tab → the failed job.
2. Expand the red step; read the last 30 lines.

3. Reproduce locally with the same command (e.g. `pytest -q`).
4. Fix, commit, push — CI runs again automatically.

15.5 Evaluation in CI (Week 7)

```
- run: python eval/run.py --set dev --out results.json
- run: python eval/compare.py results.json eval/baseline.json --fail-if "recall@5 <
-0.02"
```

16. Troubleshooting

16.1 Common errors and fixes

Message or symptom	What it means	Fix
<code>fatal: not a git repository</code>	you are not inside a repo	<code>cd</code> into the project folder
Permission denied (publickey)	SSH key not set up	Section 2.5; <code>ssh -T git@github.com</code>
<code>rejected ... (fetch first)</code>	remote has new commits	<code>git pull</code> , resolve, <code>git push</code>
<code>CONFLICT (content)</code>	same lines changed twice	Section 7.4
Your branch is ahead of 'origin/main' by 2 commits	not pushed yet	<code>git push</code>
Your branch is behind ...	teammates pushed	<code>git pull</code>
You are in 'detached HEAD' state	on a commit, not a branch	<code>git switch -c rescue</code> to keep work
error: Your local changes would be overwritten	uncommitted edits block switch/pull	commit or <code>git stash</code> first
nothing added to commit but untracked files present	forgot <code>git add</code>	<code>git add <file></code>
refusing to merge unrelated histories	two repos with separate starts	usually you cloned wrong; ask the Tech Lead
commits not on my GitHub profile	email mismatch	Section 2.2, <code>.mailmap</code>
<code>.env</code> appears in <code>git status</code>	not ignored	add to <code>.gitignore</code> ; if committed, Section 13.2
push is huge / slow	a large file was committed	Section 13.4
"every line changed" in diff	line endings	Section 2.3, <code>.gitattributes</code>

16.2 "I committed to main by mistake" (not pushed)

```
$ git branch feat/oops-work      # keep the commits on a new branch
$ git reset --hard origin/main  # put main back
$ git switch feat/oops-work      # continue here, then open a PR
```

16.3 "I made commits in detached HEAD"

```
$ git switch -c rescue-work      # the commits now belong to a branch
```

If you already switched away, find them with `git reflog` and create a branch at that hash.

16.4 "I need a file from another branch"

```
$ git restore --source=feat/rag -- src/chunker.py
```

16.5 "My branch is a mess; start over from main"

```
$ git fetch origin
$ git switch -c feat/12-fee-v2 origin/main
# copy over the good parts, or cherry-pick good commits:
$ git cherry-pick 4e5f6a7
```

16.6 "Someone force-pushed and my repo is broken"

```
$ git fetch origin
$ git branch backup-mine          # save your local state
$ git reset --hard origin/<branch> # match the remote
# re-apply your own commits from backup-mine with cherry-pick
```

Then agree as a team to protect branches and stop force-pushing.

16.7 A calm debugging routine

1. `git status` — where am I, what changed?
2. `git log --oneline --graph --all -15` — what does history look like?
3. `git stash` — protect uncommitted work.
4. Try the fix on a new branch.
5. If it goes wrong: `git reflog` — you can always go back.

17. In-Class Exercises and How to Submit

This chapter contains the exercises for Lab 1 (environment) and Lab 2 (Git collaboration and pull requests). Every exercise is marked **Individual (I)** or **Team (T)**. Teams have **four members**.

17.1 How exercises work

	Individual (I)	Team (T)
Who does it	every member, alone	all four members together
Where the work lives	your own branch and folder	shared files on main
Evidence	your commits and your PR	several PRs, reviews, a tag
Graded	per person	per team (then contribution factor)

Member numbers and reviewer rotation

At the start of Lab 2, the team fixes a member number 1–4 in `docs/team.md`. The number decides personal exercise variants and who reviews whom. The rotation changes every week so everyone reviews everyone.

Week	Member 1 reviews	Member 2 reviews	Member 3 reviews	Member 4 reviews
Week A (Lab 2, Python W1)	2	3	4	1
Week B (Python W2)	3	4	1	2
Week C (Python W3)	4	1	2	3

```
# docs/team.md
| No. | Name           | GitHub   | Role               |
|-----|-----|-----|-----|
| 1 | Nguyen Van Minh | minh-nv | Tech Lead          |
| 2 | Tran Thi Lan    | lan-tt  | Product Lead       |
| 3 | Le Quoc Huy     | huy-lq  | Knowledge & AI Engineer |
| 4 | Pham Mai        | mai-p   | QA & Security Lead  |
```

17.2 Submission rules

In-class submission (during the session)

1. Finish the exercise in the lab session.
2. Before the session ends, the evidence must be on GitHub: a pushed branch and an open (or merged) PR.
3. Submit on the course LMS (Moodle) the **link** asked for in the exercise (PR link, tag link or commit link). One link per person for Individual exercises; one link per team, submitted by the Tech Lead, for Team exercises.
4. Unfinished? Open the PR as **Draft** with a short note of what is missing. Draft PRs submitted on time receive partial credit; nothing pushed receives none.

Homework submission (between sessions)

1. Deadline: **23:59 the day before the next session**, unless the instructor announces otherwise.
2. "Submitted" means: the PR is **merged into `main`**, CI is green, and it was **approved by the assigned reviewer** (rotation table above).
3. The Tech Lead pushes the week's tag after all members have merged (for example `py-w1-done`). The instructor grades the repository **at that tag**.
4. Each member updates their row in `python-bootcamp/PROGRESS.md` (Chapter 18) with PR links.
5. Late work: commits after the tag are visible in history. Late PRs are graded only if the instructor's late policy allows.

IN OUR COURSE

The instructor never grades files sent by email or chat. If it is not in the team repository on time, it was not submitted.

Evidence file

Each week, the Tech Lead adds a short file `docs/submissions/<week>.md`:

```
# Lab 2 – submission (team 07)
Tag: v0.0-setup
Team PRs: #1 #2 #3 #4 (each reviewed by another member)
Individual:
- Minh  I1 #5  I2 #9  reviewed #6
- Lan   I1 #6  I2 #10 reviewed #7
- Huy   I1 #7  I2 #11 reviewed #8
- Mai   I1 #8  I2 #12 reviewed #5
```

17.3 Individual in-class exercises

I1 — Solo basics (30 minutes)

1. Create a local repository `git-practice-<github-username>` with `git init`.
2. Add `README.md`, `.gitignore` and `src/priority.py` in **three separate commits**.
3. Edit `priority.py`, look at `git diff`, stage only part of it with `git add -p`, and commit.
4. Delete a function on purpose, commit, then undo it with `git revert`.
5. Find the commit that created `.gitignore` with `git log -- .gitignore`.
6. Create an empty GitHub repository, add it as `origin` and push.

EXERCISE

Submit on the LMS: the repository link. The history must show at least 6 commits with Conventional Commit messages.

I2 — Undo and rescue (20 minutes)

1. Make three commits, run `git reset --hard HEAD~3`, and recover them with `git reflog`.
2. Check out an old commit (detached HEAD), make a commit, and rescue it with `git switch -c rescue`.
3. Stage a file called `.env` by mistake, unstage it, and add `.env` to `.gitignore`.

EXERCISE

Submit: a screenshot of your `git reflog` showing the recovery, and the commit link of the rescued commit.

I3 — Your first team PR (20 minutes)

1. Clone the team repository.
2. Create a branch `docs/team-<github-username>`.
3. Create `docs/team/<github-username>.md` with your name, member number, role, skills and one learning goal.
4. Commit, push and open a PR using the template. Request review from your **assigned reviewer**.
5. Review the PR you were assigned: at least one Question and one Praise comment, then approve.

EXERCISE

Submit: your PR link and the link to the PR you reviewed.

I4 — Commit message clinic (10 minutes)

Rewrite these messages in Conventional Commits style, with a one-line body explaining *why*: "fixed bug", "new stuff for rag", "Update README.md", "final version". Add your answers to `docs/team/<github-username>.md` in the same PR as I3.

17.4 Team in-class exercises**T1 — Team repository setup (40 minutes)**

Roles: Member 1 (Tech Lead) drives; everyone contributes a file.

Member	Task
1	create organisation and repo, add members, protect <code>main</code>
2	<code>CONTRIBUTING.md</code> and <code>.github/pull_request_template.md</code>
3	<code>.gitignore</code> , <code>.gitattributes</code> , <code>.env.example</code>
4	<code>.github/workflows/ci.yml</code> that runs <code>pytest</code>

Each file arrives through its own PR, reviewed by the next member in the rotation. When all four PRs are merged, Member 1 tags `v0.0-setup`.

EXERCISE

Submit (Tech Lead): the tag link. Graders check: 4 PRs, 4 different authors, 4 different approvers, CI present, `main` protected.

T2 — Conflict drill (30 minutes, two pairs)

Pair A = Members 1 & 2, Pair B = Members 3 & 4.

1. In each pair, both partners branch from the same commit.

- Both edit the same function in `src/fee.py` differently and open PRs.
- Merge the first PR. The second now conflicts.
- The second author merges `origin/main` into their branch and resolves the conflict **together** with the partner, runs the tests and pushes.
- The merge commit message explains the final decision.

EXERCISE

Submit (Tech Lead): the two merge-commit links, one per pair.

T3 — Review an AI-generated PR (25 minutes)

The instructor provides a sample PR with three planted problems (for example: an unnecessary factory class, an off-by-one in a date range, and an API key in the code).

- Each member reviews the PR independently for 10 minutes and writes comments labelled Blocking / Suggestion / Question.
- The team compares comments for 10 minutes and agrees which problem must block the merge first.
- Member 4 (QA & Security Lead) writes a summary comment with the team decision.

EXERCISE

Submit (Tech Lead): the link to the summary comment. Every member must have at least one comment on the PR.

T4 — Contribution report (20 minutes)

- Add `.mailmap` if anyone committed under two identities.
- Run `git shortlog -sne --no-merges` and `tools/contrib_report.py` for the lab.
- Count PRs and reviews per person with `gh` (Section 12.5).
- Compare with `docs/team/` and the logbook. Write three sentences in `docs/contrib/lab2.md`: what the numbers show, what they miss, and one change for next week.

EXERCISE

Submit (Tech Lead): the link to `docs/contrib/lab2.md` on `main`.

17.5 How exercises are checked

Check	Individual	Team
Work is on GitHub before the deadline	✓	✓
Commits under your own linked email	✓	✓
Conventional Commit messages	✓	✓
PR reviewed by the assigned reviewer	✓	✓
Every member visible in PRs or reviews	—	✓

Check	Individual	Team
Tag pushed by the Tech Lead	—	✓
No secrets or personal data	✓	✓

18. Python Bootcamp: Three Weeks of Homework, Tracked on GitHub

You already know programming from C++. The team project is written in Python. This chapter is a three-week homework plan that teaches Python **by comparison with C++**, while practising the Git workflow from the previous chapters. Every exercise is submitted through your team repository, so each member's progress and contribution is visible.

18.1 Overview

Week	Python topics	Individual homework	Team homework	Tag
1	syntax, types, control flow, functions, lists, dicts, strings	W1-1 ... W1-5	T-W1: bootcamp folder + CI	py-w1-done
2	modules, files, CSV/JSON, exceptions, classes, dataclasses, pytest	W2-1 ... W2-5	T-W2: shared Task contract, one function each	py-w2-done
3	datetime, collections, typing, argparse, ruff, packaging	W3-1 ... W3-4	T-W3: integrate a study_planner CLI	py-w3-done

Expected effort: about **4 hours of individual work** and **1 hour of team work** per week.

IN OUR COURSE

The bootcamp is graded per person (individual exercises, reviews given) and per team (team exercises). Your GitHub activity inside [python-bootcamp/](#) is the evidence.

18.2 Repository layout (team setup, do once)

Member 1 creates this structure in the team repository in Week 1 (exercise T-W1):

```
python-bootcamp/
├── README.md           # rules, how to run tests
├── PROGRESS.md         # one row per member per week (links to PRs)
├── members/
│   ├── minh_nv/       # one folder per member: GitHub username,
│   │   ├── w1/ w2/ w3/ #   hyphens replaced by underscores
│   │   ├── lan_tt/
│   │   ├── huy_lq/
│   │   └── mai_p/
│   ├── shared/        # team code (T-W2, T-W3)
│   │   ├── contract.py
│   │   └── study_planner/
│   └── tests/         # tests provided by the instructor
│       ├── test_w1.py test_w2.py test_w3.py
```

Folder names use underscores (`minh_nv`, not `minh-nv`) because Python cannot import a module whose name contains a hyphen. Each member writes code **only in their own folder** for individual exercises. This avoids conflicts and makes authorship obvious.

CI for the bootcamp

Add a job to `.github/workflows/ci.yml` (Chapter 15):

```
bootcamp:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
      with: {python-version: "3.12"}
    - run: pip install pytest ruff
    - run: ruff check python-bootcamp
    - run: pytest -q python-bootcamp
```

Tracking setup

1. Create GitHub **milestones** `PY-W1`, `PY-W2`, `PY-W3` with the homework deadlines.
2. Create labels `python-hw`, `w1`, `w2`, `w3`, `team-hw`.
3. Each week, each member opens **one issue** for their homework using the template below and assigns it to themselves.

```
---
name: Python homework
about: Weekly individual Python exercises
labels: python-hw
---
Week: w1
Member: @minh-nv (No. 1)
Reviewer (rotation): @lan-tt
- [ ] W1-1 - [ ] W1-2 - [ ] W1-3 - [ ] W1-4 - [ ] W1-5
Estimated hours:   Actual hours:
```

PROGRESS.md

Member	Week	Issue	PR	Reviewer	Tests	Hours
minh-nv	w1	#21	#25	lan-tt	18/18	4.5
lan-tt	w1	#22	#26	huy-lq	18/18	4
huy-lq	w1	#23	#27	mai-p	16/18	5
mai-p	w1	#24	#28	minh-nv	18/18	3.5

18.3 The weekly workflow for every member

1. Pull `main`, then create your branch: `git switch -c py/w1-<github-username>`.
2. Do one exercise at a time. **Commit after each exercise** (at least 5 commits per week), with messages like `feat(py-w1): W1-2 word counter`.
3. Run the tests for your folder often:

```
$ pytest -q python-bootcamp/tests/test_w1.py --member minh_nv --variant 1
```

4. Push daily: `git push -u origin py/w1-minh-nv`.
5. Open a PR titled `py-w1: minh-nv`, link your issue (Closes #21), and request your rotation reviewer.
6. Review the PR you were assigned (Section 18.9). Approve only after the tests pass and you understand the code.
7. Merge after approval. Update your row in `PROGRESS.md` in the same PR or a follow-up.

TIP

The `--member` and `--variant` (your member number, for W1-4) options are provided by `python-bootcamp/tests/conftest.py`, which the instructor supplies together with `samples/` and a README listing the exact function names. It imports your modules from `members/<your_folder>/wN/`.

18.4 From C++ to Python: a quick guide

The same program in both languages

```
// C++
#include <iostream>
#include <vector>
int main() {
    std::vector<int> xs = {3, 1, 2};
    int total = 0;
    for (int x : xs) total += x;
    std::cout << "total = " << total << "\n";
}
```

```
# Python
xs = [3, 1, 2]
total = 0
for x in xs:
    total += x
print(f"total = {total}")
# or simply: print(f"total = {sum(xs)}")
```

Syntax mapping

C++	Python	Note
<code>int x = 5;</code>	<code>x = 5</code>	no declarations; types are dynamic
<code>{ ... }</code> blocks	indentation (4 spaces)	indentation is syntax
<code>if (a && b)</code>	<code>if a and b:</code>	and, or, not
<code>for (int i = 0; i < n; i++)</code>	<code>for i in range(n):</code>	<code>range(start, stop, step)</code>
<code>while (x > 0)</code>	<code>while x > 0:</code>	
<code>std::vector<int></code>	list	<code>xs.append(4), len(xs)</code>
<code>std::map<string, int></code>	dict	<code>d["a"] = 1, d.get("b", 0)</code>
<code>std::set<int></code>	set	<code>{1, 2, 3}</code>

C++	Python	Note
<code>std::pair, std::tuple</code>	<code>tuple</code>	<code>(1, "a")</code> , unpacking <code>a, b = t</code>
<code>struct Task {...}</code>	<code>@dataclass class Task:</code>	Week 2
<code>std::string s; s.size()</code>	<code>s = "..."; len(s)</code>	strings are immutable
<code>7 / 2 → 3</code>	<code>7 // 2 → 3, 7 / 2 → 3.5</code>	<code>/</code> is always float division
<code>nullptr</code>	<code>None</code>	
<code>try { } catch (...) { }</code>	<code>try: ... except ValueError:</code>	Week 2
<code>#include <x></code>	<code>import x</code>	modules, Week 2
<code>cin >> x</code>	<code>x = int(input())</code>	input is always a string

Things that surprise C++ programmers

- **No compile step.** Errors appear when the line runs. Tests matter even more.
- **Everything is an object; variables are names.** `b = a` for a list does *not* copy it: both names point to the same list. Use `b = a.copy()` or `list(a)`.
- **Integers never overflow.** `2 ** 100` just works.
- **Default mutable arguments are shared** — never write `def f(xs=[])`; use `xs=None`.
- **Truthiness.** Empty list, empty string, `0` and `None` are all false in `if`.
- **Iteration is idiomatic:** `for i, x in enumerate(xs), for a, b in zip(xs, ys)`.

Comprehensions

```
squares = [x * x for x in range(10) if x % 2 == 0]      # [0, 4, 16, 36, 64]
by_len   = {w: len(w) for w in ["git", "python"]}      # {'git': 3, 'python': 6}
```

18.5 Week 1 — Basics

Topics to study

Variables and types, `if/for/while`, functions and default arguments, lists, tuples, dicts, sets, strings and f-strings, slicing, comprehensions, `input()/print()`. Suggested reading: the official Python tutorial, sections 3–5.

W1-1 Grade summary (members/<you>/w1/grades.py)

Write `summary(scores: list[float]) -> dict` returning `{"min", "max", "mean", "median"}`. Round mean and median to 2 decimals. An empty list raises `ValueError`.

```
>>> summary([7.5, 9, 6, 8])
{'min': 6, 'max': 9, 'mean': 7.62, 'median': 7.75}
```

W1-2 Word counter (text_tools.py)

Write `word_count(text: str) -> dict[str, int]` (lower-case, ignore punctuation `., !? ; :`) and `top_k(text: str, k: int) -> list[tuple[str, int]]` sorted by count, then alphabetically.

```
>>> top_k("Git is fun. Git is fast!", 2)
[('git', 2), ('is', 2)]
```

W1-3 Thesis eligibility (rules.py)

Write `can_register_thesis(credits: int, gpa: float) -> bool`: true if credits ≥ 120 and gpa ≥ 2.0 . Then `missing(credits, gpa) -> list[str]` returning human-readable reasons, e.g. `["need 2 more credits"]`.

W1-4 Translate C++ to Python (translate.py) — personal variant

Each member translates a different C++ function. Keep the same behaviour, write it the *Pythonic* way, and add a docstring explaining one difference from the C++ version.

Member	C++ function to translate
1	binary search on a sorted vector $\rightarrow$ return index or -1
2	insertion sort (in place) $\rightarrow$ return the sorted list
3	transpose of a matrix (vector of vectors)
4	sieve of Eratosthenes $\rightarrow$ primes up to n

```
// Member 1 – C++ starting point
int binary_search(const std::vector<int>& a, int key) {
    int lo = 0, hi = (int)a.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == key) return mid;
        if (a[mid] < key) lo = mid + 1; else hi = mid - 1;
    }
    return -1;
}
```

W1-5 Group the timetable (timetable.py)

Given a list of `(course, day)` tuples, return a dict from day to a sorted list of courses, using a comprehension or `dict.setdefault`.

```
>>> by_day([("CSC10014", "Mon"), ("MTH00003", "Tue"), ("CSC10001", "Mon")])
{'Mon': ['CSC10001', 'CSC10014'], 'Tue': ['MTH00003']}
```

Team T-W1 — Bootcamp setup (1 hour)

Member	Task (one PR each)
1	create <code>python-bootcamp/</code> structure and <code>README.md</code>
2	milestones, labels, issue template
3	CI job for the bootcamp
4	<code>PROGRESS.md</code> and copy the instructor's <code>tests/</code>

EXERCISE

Week 1 is complete when: four individual PRs (`py-w1: <username>`) are merged, each approved by the rotation reviewer; CI is green; `PROGRESS.md` has four rows; Member 1 has pushed tag `py-w1-done`.

18.6 Week 2 — Modules, files, errors and classes

Topics to study

Modules and packages (`import`, `__init__.py`, `if __name__ == "__main__":`), reading and writing files (`with open(...)`), `csv` and `json`, exceptions and custom exception classes, classes, `@dataclass`, virtual environments (`python -m venv .venv`), `pip`, and writing tests with `pytest`.

C++ → Python: structs and classes

```
// C++
struct Task {
    std::string title;
    int importance;
    double effort_h;
};
```

```
# Python
from dataclasses import dataclass

@dataclass
class Task:
    title: str
    importance: int
    effort_h: float

    def __post_init__(self):
        if not 1 <= self.importance <= 5:
            raise ValueError("importance must be 1-5")
```

W2-1 Task model (`members/<you>/w2/models.py`)

Implement the `Task` dataclass above plus a `deadline: date` field and a method `days_left(today: date) -> int` (never below 0).

W2-2 Load and save (`io_utils.py`)

Write `load_tasks(path) -> list[Task]` reading a CSV with header `title,importance,effort_h,deadline`, and `save_json(tasks, path)`. Invalid rows raise `ValueError` with the line number.

```
title,importance,effort_h,deadline
DS exam,5,6,2026-10-30
Assignment,4,1.5,2026-10-27
```

W2-3 Custom exception (errors.py)

Create `class InfeasiblePlan(Exception)` carrying `needed_h` and `available_h`, with a clear `__str__`. Write `check_feasible(tasks, available_h)` that raises it.

W2-4 Test someone else's code (tests_for_<reviewee>.py)

Write **at least five pytest tests** for the W1 functions of the member you review this week (rotation Week B). Include one edge case the instructor's tests do not cover. Open this in the same PR as your W2 work; the author must respond if a test fails.

NOTE

W2-4 makes reviewing concrete: you read a teammate's code closely enough to test it. It also creates cross-member contribution evidence.

W2-5 A small package

Turn your `w2/` folder into a package (`__init__.py`) and add `python -m members.<you>.w2` that loads the sample CSV and prints the tasks.

Team T-W2 — Shared contract, one function each (1–2 hours)

First, the team agrees on `shared/contract.py` in **one PR reviewed by all four members**:

```
from dataclasses import dataclass
from datetime import date

@dataclass
class Task:
    title: str
    importance: int      # 1-5
    effort_h: float
    deadline: date

@dataclass
class Block:
    task: Task
    hours: float
```

Then each member implements **one function** in `shared/study_planner/`, with tests, in their own PR:

Member	Module	Function
1	<code>parse.py</code>	<code>parse_csv(path) -> list[Task]</code>
2	<code>prioritise.py</code>	<code>prioritise(tasks, today) -> list[Task]</code> (importance ÷ days_left)
3	<code>allocate.py</code>	<code>allocate(tasks, hours) -> list[Block]</code> (greedy)
4	<code>render.py</code>	<code>render(blocks) -> str</code> (a readable plan)

EXERCISE

Week 2 is complete when: four individual PRs are merged; the contract PR shows approvals from all four members; four module PRs are merged with tests; tag `py-w2-done` is pushed.

18.7 Week 3 — Standard library and tooling

Topics to study

`datetime` and `date` arithmetic, `pathlib`, `collections` (`Counter`, `defaultdict`), `itertools`, type hints, `argparse` for command-line tools, `logging`, code style with `ruff`, and reading documentation.

W3-1 Dates (`members/<you>/w3/dates.py`)

Write `parse_due(text: str, today: date) -> date` accepting `"2026-10-30"`, `"tomorrow"`, `"in 3 days"` and weekday names like `"Fri"` (next occurrence). Unknown text raises `ValueError`.

W3-2 Chat log analysis (`log_stats.py`)

Write `analyse(pairs) -> dict` that, given a list of `(route, latency_ms)` pairs, returns `{"counts": ..., "p95": ..., "slowest": ...}`: requests per route (`Counter`), p95 latency per route, and the route with the highest p95.

W3-3 A command-line tool (`cli.py`)

```
$ python -m members.minh_nv.w3.cli tasks.csv --hours 2 --today 2026-10-26
Tonight (2.0 h):
  1. Assignment ..... 1.5 h
  2. DS exam ..... 0.5 h
```

Use `argparse`, reuse your W2 code, accept an optional `--today YYYY-MM-DD` (default: today) so results are reproducible, and exit with code 1 and a clear message if the file is missing.

W3-4 Clean code

Add type hints to all your bootcamp functions, make `ruff check` pass, and add docstrings. Write three lines in your PR describing the Python feature you found most different from C++.

Team T-W3 — Integrate the study planner (2 hours)

1. Member 1 writes `shared/study_planner/__main__.py` that calls `parse_csv` → `prioritise` → `allocate` → `render`.
2. Member 2 writes an end-to-end test with a sample CSV.
3. Member 3 adds error handling: missing file, empty file, infeasible plan (reuse `InfeasiblePlan`).
4. Member 4 writes `shared/study_planner/README.md` and a short demo in `docs/`.

```
$ python -m shared.study_planner samples/tasks.csv --hours 2
```

EXERCISE

Week 3 is complete when: four individual PRs are merged; the CLI runs from a fresh clone; the end-to-end test passes in CI; tag `py-w3-done` is pushed.

18.8 Tracking progress and contributions

Per member, per week

```
$ git shortlog -sn --no-merges py-w1-done..py-w2-done -- python-bootcamp/members/
$ git log --no-merges --oneline --author="Lan" -- python-bootcamp/
$ gh pr list --state merged --label w2 --json author,number,title
$ gh search prs --repo csc10014-team07/assistant-team07 --label w2 --reviewed-by huy-lq --json
number | jq length
```

Per folder: who wrote what

```
$ for m in python-bootcamp/members/*/; do
  echo "== $m"; git shortlog -sn --no-merges -- "$m"; done
```

Only the owner should appear in their own folder. Anyone else appearing there is worth a conversation (pair work should use **Co-authored-by**).

Team code

```
$ git shortlog -sn --no-merges -- python-bootcamp/shared/
$ python tools/contrib_report.py py-w2-done..py-w3-done
```

What the instructor looks at

Evidence	Where
Individual exercises done	merged py-wN : <username> PR, tests passing
Steady work	at least 5 commits per week, spread over several days
Reviews given	approvals and comments on the assigned reviewee's PR
Cross-testing (W2-4)	tests written for a teammate's code
Team exercises	contract PR approvals, one module PR per member
Honest progress	PROGRESS.md hours and links match GitHub

18.9 Reviewing a teammate's Python homework

Use this checklist when you review the PR assigned to you:

- Do the instructor's tests pass in CI?
- Is the code Pythonic (loops over items, comprehensions, f-strings) — or C++ written in Python syntax?
- Are edge cases handled (empty input, wrong types, zero)?
- Are names clear and are there docstrings?
- Leave at least **one Question, one Suggestion and one Praise**.

18.10 Homework grading (proposal)

Criterion	Weight
Individual exercises correct (tests)	50%
Code quality (Pythonic, typed, ruff clean)	15%
Git practice (small commits, messages, PR description)	15%
Reviews and cross-tests given	10%
Team exercise contribution	10%

18.11 Integrity and AI use

- Write the individual exercises **yourself**. Personal variants (W1-4) and cross-tests (W2-4) make copying easy to spot.
- You may use AI tools to *explain* a concept or an error message. If AI generated any code you kept, say so in the PR "AI use" section and in `docs/ai-use-log.md`.
- You must be able to explain every line of your homework in the individual defense.

CHECK YOURSELF

- 1) Why does each member write code only in `members/<username>/`?
- 2) Which two things must be true before your weekly PR can be merged?
- 3) Which command shows who committed in the `shared/` folder during Week 3?

19. Git Under the Hood

You can use Git for years without knowing its internals. But ten minutes here will make every other command less mysterious.

19.1 Four kinds of objects

Git stores everything in `.git/objects` as four object types, each named by the hash of its content:

Object	Stores	Analogy
blob	the content of one file (no name)	a page of text
tree	a folder: names → blobs and sub-trees	a table of contents
commit	one tree + parent(s) + author + message	a dated snapshot label
tag (annotated)	a name + message pointing to a commit	a bookmark with a note

19.2 Look inside with cat-file

```
$ git cat-file -t HEAD
commit
$ git cat-file -p HEAD
tree 9a3c5e1f...
parent d4c7a90b...
author Tran Thi Lan <lan.tt@student.hcmus.edu.vn> 1759752843 +0700
committer Tran Thi Lan <lan.tt@student.hcmus.edu.vn> 1759752843 +0700

feat(fee): add retake rate

$ git cat-file -p 9a3c5e1f
100644 blob 5b1d3e7a...    README.md
040000 tree 2c7f9a10...    src
040000 tree 7e8d1c44...    tests
```

19.3 Branches are tiny files

```
$ cat .git/HEAD
ref: refs/heads/main
$ cat .git/refs/heads/main
4e5f6a7c2d1b...
```

A branch is literally a file containing one commit hash. That is why creating a branch is instant, and why "deleting a branch" never deletes commits by itself — it deletes the label.

19.4 Why identical files cost nothing

If ten commits contain the same `README.md`, Git stores the blob once; all ten trees point to it. Git also packs similar objects together with delta compression (`git gc`), so repositories stay small.

19.5 What this explains

Behaviour	Explanation
You cannot edit an old commit silently	changing content changes the hash, and every later hash
<code>rebase</code> creates "new" commits X', Y'	new parents → new content → new hashes
Lost commits can be recovered	objects stay until garbage collection; <code>reflog</code> remembers them
Cloning gives you everything	every object is copied — every clone is a full backup

NOTE

Git is migrating from SHA-1 to SHA-256 hashes. For everyday use nothing changes: you still refer to commits by the first 7–10 characters of their hash.

20. Git in VS Code

Most students write code in VS Code, which has a solid Git interface. Use it for speed — but always know which command it runs.

20.1 The Source Control panel

1. Open the project folder (*File* → *Open Folder*). VS Code detects the `.git` folder.
2. Click the **Source Control** icon (Ctrl+Shift+G). Changed files appear under *Changes*.
3. Click a file to see a side-by-side diff.
4. Click + next to a file to stage it (`git add`). Right-click lines in the diff to stage single lines.
5. Type a message in the box and press **Commit** (Ctrl+Enter).
6. Click **Sync Changes** or use the ... menu for Pull / Push.

VS Code action	Git command
+ (Stage Changes)	<code>git add <file></code>
– (Unstage)	<code>git restore --staged <file></code>
Discard Changes	<code>git restore <file></code>
Commit	<code>git commit -m</code>
Branch name in status bar	<code>git switch / git switch -c</code>
Sync	<code>git pull</code> then <code>git push</code>

WARNING

"Discard Changes" and "Discard All Changes" delete uncommitted work without a recycle bin. Stash first if unsure.

20.2 Resolving conflicts in VS Code

When a merge conflicts, VS Code highlights each conflict with buttons: **Accept Current Change**, **Accept Incoming Change**, **Accept Both Changes**, **Compare Changes**. For larger conflicts, use the **Resolve in Merge Editor** button, which shows *Incoming*, *Current* and *Result* panes.

After resolving, stage the file and commit the merge as usual. Run the tests first.

20.3 Helpful extensions

- **GitLens** — inline blame, file history, and a visual commit graph.
- **GitHub Pull Requests** — review and comment on PRs inside VS Code.
- **Git Graph** — a clickable history graph.

TIP

Enable *Settings* → *Git: Confirm Sync* and *Git: Always Sign Off* only if your team agrees. Keep defaults otherwise.

21. Issues, Projects and Milestones in Depth

Git records *what* changed. GitHub's planning tools record *why*, *who* and *when*. Used well, they make weekly reviews and contribution tracking effortless.

21.1 Writing a good issue

Part	Good example
Title	"Router: ask a clarifying question when deadline is missing"
Context	links to UC-01 and the failed benchmark case B-017
Acceptance criteria	checklist of observable behaviours
Size	fits in 1–2 days; otherwise split
Labels	<code>feat</code> , <code>router</code> , <code>M4</code>
Assignee	one owner (others can help)

21.2 Labels that help this course

Label	Use
<code>feat</code> , <code>fix</code> , <code>docs</code> , <code>test</code> , <code>chore</code>	type of work
<code>router</code> , <code>rag</code> , <code>tools</code> , <code>ui</code> , <code>eval</code>	component
<code>blocked</code>	waiting for someone or something
<code>good-first-issue</code>	small task for a new or quieter member
<code>security</code> , <code>privacy</code>	needs extra review (Week 6)

21.3 The project board

Create a *Project* (table or board view) with columns **Todo** → **In progress** → **In review** → **Done**. Add fields for *Milestone*, *Size* and *Component*. Rules of thumb:

- An issue moves to *In progress* only when someone is actually working on it.
- *In review* means a PR is open.
- Limit work in progress: at most two issues per person in *In progress*.

21.4 Milestones

Create GitHub milestones M1–M8 with the course due dates. Assign every issue and PR to one. The milestone page then shows percentage complete — a quick health check before each review.

21.5 Linking everything

```
Issue #27 "Router: clarify missing deadline"
└─ branch feat/27-clarify-deadline
```

```
└─ commits "feat(router): ..." (Refs #27)
   └─ PR #35 "Closes #27" → reviewed by @kenji-s → merged
      └─ milestone M4, tag v0.4-prototype
```

With these links, anyone can go from a line of code to the commit, the PR discussion, the issue and the use case — which is exactly the traceability the rubric rewards.

21.6 GitHub CLI for planning

```
$ gh issue create --title "Router: clarify missing deadline" --label feat,router --milestone M4 --assignee @me
$ gh issue list --assignee @me --state open
$ gh issue view 27 --comments
$ gh pr status
```

EXERCISE

Create milestones M1–M8, a project board with four columns, five labels, and four issues for your current sprint — one per member.

22. Tricky Files and Special Cases

22.1 Jupyter notebooks

Notebooks store outputs and metadata as JSON, which creates noisy diffs and painful conflicts.

- Clear outputs before committing (*Kernel* → *Restart & Clear Output*), or use `nbstripout`:

```
$ pip install nbstripout
$ nbstripout --install
```

- Keep real logic in `.py` modules; use notebooks only for exploration.
- Use `nbdime` for readable notebook diffs: `nbdiff-web`.

22.2 JSON, YAML and lock files

- Keep configuration files sorted and formatted consistently to reduce conflicts.
- For lock files (`uv.lock`, `package-lock.json`), do not hand-merge: take one side, then regenerate with the package manager and commit the result.

22.3 Binary files (images, PDFs, docx)

Git cannot merge binary files. If two people edit the same PDF or image, one version wins.

- Agree on a single owner for each binary document.
- Prefer text formats: Markdown for reports, Mermaid or PlantUML for diagrams.
- Export final PDFs only at milestones.

```
$ git checkout --theirs docs/architecture.png # keep the incoming version
$ git checkout --ours docs/architecture.png   # keep your version
$ git add docs/architecture.png
```

22.4 Case-sensitive file names

Windows and macOS usually ignore letter case; Linux (and CI) do not. Renaming `Readme.md` to `README.md` needs:

```
$ git mv Readme.md README.tmp
$ git mv README.tmp README.md
```

22.5 Empty folders

Git tracks files, not folders. To keep an empty folder, add a placeholder such as `data/.gitkeep`.

22.6 Submodules — avoid unless needed

Submodules embed another repository inside yours. They are powerful but confusing for beginners. For this course, prefer installing shared code as a package or copying small utilities with attribution.

EXERCISE

Add `nbstripout` to your repository, commit a notebook with outputs, and check that the committed version has no outputs. Then create an empty `data/processed/` folder with a `.gitkeep` file.

Appendix A — Command Cheat Sheet

Task	Command
set identity	<code>git config --global user.name / user.email</code>
new repo	<code>git init</code>
copy repo	<code>git clone <url></code>
what changed?	<code>git status, git diff, git diff --staged</code>
stage	<code>git add <file>, git add -p</code>
commit	<code>git commit -m "type(scope): summary"</code>
history	<code>git log --oneline --graph --all</code>
one commit	<code>git show <hash></code>
who wrote a line	<code>git blame -w -C <file></code>
search history	<code>git log -S "text", git log --grep="word"</code>
new branch	<code>git switch -c <name></code>
switch branch	<code>git switch <name></code>
merge	<code>git merge <branch></code>
rebase own branch	<code>git rebase origin/main</code>
download	<code>git fetch, git pull</code>
upload	<code>git push -u origin <branch></code>
discard file edits	<code>git restore <file></code>
unstage	<code>git restore --staged <file></code>
fix last commit	<code>git commit --amend</code>
undo pushed commit	<code>git revert <hash></code>
park work	<code>git stash, git stash pop</code>
rescue	<code>git reflog</code>
tag milestone	<code>git tag -a v0.5-mvp -m "...", git push origin v0.5-mvp</code>
contributions	<code>git shortlog -sne --no-merges</code>
open PR	<code>gh pr create</code>
check out PR	<code>gh pr checkout <n></code>

Appendix B — Glossary

Term	Meaning
branch protection	repository rules that block direct pushes or unreviewed merges
cherry-pick	copy one commit onto the current branch
CI	continuous integration: automatic checks on every change
clone	a full local copy of a remote repository
commit	a saved snapshot with author, date, message and parent
conflict	two changes to the same lines that Git cannot combine alone
Conventional Commits	a message format: <code>type(scope): summary</code>
Co-authored-by	a trailer that credits extra authors of a commit
fast-forward	a merge that only moves the branch label
fork	your own GitHub copy of someone else's repository
HEAD	the commit or branch you are currently on
index / staging area	the draft of the next commit
mailmap	a file that merges multiple identities of one person
merge commit	a commit with two parents
origin	the default name of the main remote
pull request	a GitHub request to merge a branch, with discussion and review
rebase	replay commits on top of another base
reflog	a local log of every position of HEAD
remote	another copy of the repository, e.g. on GitHub
revert	a new commit that undoes an earlier one
squash	combine several commits into one
stash	a temporary shelf for uncommitted changes
tag	a permanent name for one commit, e.g. a milestone
upstream	the remote branch your local branch tracks

Appendix C — Templates

C.1 CONTRIBUTING.md

```
# Contributing to assistant-team07

## Workflow
1. Every change starts as an issue on the board.
2. Branch from main: feat/<issue>-<short-name> (fix/, docs/, test/, chore/).
3. Small commits in Conventional Commits style.
4. Open a PR using the template; link the issue ("Closes #12").
5. At least 1 approval from another member; CI must be green.
6. Squash and merge; delete the branch.

## Rules
- Never push to main. Never force-push shared branches.
- No secrets, no real personal data, no files > 5 MB.
- Review requested PRs within 24 hours.
- Record AI use in the PR and in docs/ai-use-log.md.
- Pair work: add Co-authored-by trailers.
```

C.2 Pull request template (.github/pull_request_template.md)

```
## What
<one or two sentences>

## Why
Closes #<issue>. <link to use case / ADR>

## How tested
- [ ] unit tests: <command and result>
- [ ] manual check: <steps>

## AI use
<tool, what it produced, what you changed, how you verified – or "None">

## Checklist
- [ ] small and focused (< ~300 lines)
- [ ] no secrets or personal data
- [ ] docs / README updated if needed
```

C.3 Issue template (.github/ISSUE_TEMPLATE/task.md)

```
---
name: Task
about: A unit of work for the sprint
---
## Goal
<user-visible outcome, link to use case>

## Acceptance criteria
- [ ] ...
- [ ] ...
```

```
## Notes
<data, interfaces, risks>
```

C.4 CODEOWNERS (.github/CODEOWNERS)

```
# Reviews are requested automatically from these owners
/src/router/    @minh-nv
/src/tools/     @lan-tt
/src/rag/       @huy-lq
/eval/         @mai-p
/docs/         @mai-p
```

C.5 .gitignore for the assistant project

```
.venv/
__pycache__/
*.pyc
.pytest_cache/
.ruff_cache/
.env
.env.*
!.env.example
data/raw/
data/private/
*.sqlite
*.db
.DS_Store
.vscode/
.idea/
node_modules/
dist/
```

Appendix D — Self-Check Quiz

Try the questions first; answers follow.

1. What are the three areas of Git, and which command moves changes between each pair?
2. You committed a typo in the message and have not pushed. How do you fix it?
3. You pushed a commit that broke `main`. Which command undoes it safely, and why not `reset`?
4. What exactly is a branch?
5. What is the difference between `git fetch` and `git pull`?
6. Two teammates changed the same line. What does Git do, and what must you do?
7. Why must your Git email match your GitHub email?
8. Name three sources of contribution evidence besides commit counts.
9. You accidentally committed an API key and pushed. List the first two actions in order.
10. When is `rebase` dangerous?
11. What does `Co-authored-by` do?
12. What should a PR description contain in this course?

Answers

1. Working directory → staging area (`git add`) → repository (`git commit`); back with `git restore` (and `git restore --staged`).
2. `git commit --amend -m "new message"`.
3. `git revert <hash>` — it adds a new undoing commit; `reset` rewrites shared history and breaks teammates' clones.
4. A movable label (pointer) to a commit.
5. `fetch` only downloads; `pull` downloads and merges into your current branch.
6. Git stops with a conflict and marks the file; you edit the file to the correct result, remove the markers, test, `git add`, and commit.
7. Otherwise GitHub cannot link the commits to your account and your contribution looks missing.
8. Any three of: merged pull requests, reviews and review comments, issues, docs/ADRs, benchmark cases, co-authored commits, the team logbook.
9. Revoke/rotate the key at the provider; then remove it from the code (and history if needed) and inform the team.
10. When the branch is shared with others (e.g. `main`) — rebasing rewrites commits they already have.
11. It credits additional authors of a commit; GitHub shows and counts it for them.
12. What, why (linked issue), how it was tested, AI use, and the checklist.

Further Reading

- *Pro Git* by Scott Chacon and Ben Straub — free online at git-scm.com/book.
- GitHub Docs — "Hello World", "About pull requests", "About protected branches", "GitHub Actions quickstart".
- MIT *The Missing Semester of Your CS Education* — lecture on version control (Git).
- Google Engineering Practices — "What to look for in a code review" and "Small CLs".
- Conventional Commits specification — conventionalcommits.org.
- *Oh My Git!* — an open-source game for learning Git visually.